

```
1 var _diagMsgs = [];
2 var _diagStart = Date.now();
3 function _diag(msg) {
4   var ts = Date.now() - _diagStart;
5   var line = '[' + ts + 'ms] ' + msg;
6   _diagMsgs.push(line);
7   try { console.log(line); } catch(_) {}
8 }
9 function _showDiag() {
10  try {
11    if (typeof wx !== 'undefined' && wx.showModal) {
12      wx.showModal({
13        title: '启动诊断',
14        content: _diagMsgs.join('\n'),
15        showCancel: false
16      });
17    }
18  } catch(_) {}
19 }
20 _diag('game.js 开始执行');
21 try {
22   if (typeof wx !== 'undefined') {
23     var _si = wx.getSystemInfoSync();
24     _diag('platform:' + _si.platform + ' system:' + _si.system);
25     _diag('brand:' + _si.brand + ' model:' + _si.model);
26   }
27 } catch(e) {
28   _diag('getSystemInfo失败:' + e);
29 }
30 try {
31   if (typeof GameGlobal !== 'undefined') {
32     GameGlobal.onError = function(msg) {
33       _diag('onError:' + msg);
34       _showDiag();
35     };
36     GameGlobal.onUnhandledRejection = function(ev) {
37       _diag('unhandledRej:' + (ev && ev.reason || ev));
38       _showDiag();
39     };
40   }
41 } catch(_) {}
42 // —— xiao_chu 方式: 把"主屏 canvas"提前锁定为 2D ctx 用作合成上屏 ——
43 // 微信小游戏里同一个 canvas 只能持有一种 ctx 类型 (webgl 或 2d) , 且必须在
44 // 拿过 ctx 之后才会"上屏"--所以必须在 pixi-adapter 接管之前就把第一次
45 // wx.createCanvas() 的主 canvas 立刻 getContext('2d') 锁定, 否则 iOS 真机上
46 // 之后再拿去 2D ctx 会失败, 导致好友榜 sharedCanvas 无法合成上屏。
47 // 把它挂到 GameGlobal.__mainCanvas / __mainCtx2d, 让 Game.ts 走双 canvas 模式。
48 try {
49   if (typeof wx !== 'undefined' && typeof wx.createCanvas === 'function'
50     && typeof GameGlobal !== 'undefined' && !GameGlobal.__mainCanvas) {
```

```
51   var _mainCanvas = wx.createCanvas();
52   var _mainCtx2d = null;
53   try {
54     _mainCtx2d = _mainCanvas.getContext('2d');
55   } catch (e) {
56     _diag('main getContext(2d) 失败:' + e);
57   }
58   GameGlobal.__mainCanvas = _mainCanvas;
59   GameGlobal.__mainCtx2d = _mainCtx2d;
60   _diag('xiao_chu mainCanvas ready ctx2d=' + (_mainCtx2d ? 'ok' : 'null'));
61 }
62 } catch (e) {
63   _diag('xiao_chu mainCanvas init failed:' + e);
64 }
65 _diag('加载 pixi-adapter...');
66 try {
67   require('./pixi-adapter/index');
68   _diag('pixi-adapter OK');
69 } catch (e) {
70   _diag('pixi-adapter 失败!!!' + e);
71   _showDiag();
72 }
73 if (typeof Intl === 'undefined') {
74   _diag('Intl不存在,注入polyfill');
75   var _g = typeof GameGlobal !== 'undefined' ? GameGlobal : (typeof globalThis !== 'undefined' ? ...
76   _g.Intl = {};
77 }
78 _diag('加载 game-bundle...');
79 try {
80   require('./game-bundle.js');
81   _diag('game-bundle OK');
82 } catch (e) {
83   _diag('game-bundle 失败!!!' + e);
84   _showDiag();
85 }
86 _diag('全部加载完成');
87 setTimeout(function() {
88   if (typeof GameGlobal !== 'undefined' && !GameGlobal.__gameRendered) {
89     _diag('5秒超时 - 游戏未渲染');
90     _showDiag();
91   }
92 }, 5000);
93 import { Analytics, EVENT_NAMES, type DeviceInfo, type EventParamValue, type PlatformName } from ...
94 import { BACKEND_BASE_URL, GAME_KEY } from '@config/CloudConfig';
95 import { Platform } from '@core/PlatformService';
96 type AnalyticsParams = Record<string, EventParamValue>;
97 export interface AnalyticsAdContext {
98   adUnitId: string;
99   scene: string;
100   adType: string;
```

```
101  levelId?: number | string;
102  extra?: Record<string, string | number | boolean>;
103  }
104  type LevelEvent = {
105    levelId: number | string;
106    levelName?: string;
107    mode?: string;
108    durationMs?: number;
109    reason?: string;
110    extra?: Record<string, string | number | boolean>;
111  };
112  type TutorialEvent = {
113    stepId: string;
114    stepIndex?: number;
115    status?: string;
116    isForce?: boolean;
117    extra?: Record<string, string | number | boolean>;
118  };
119  function track(eventName: string, params: AnalyticsParams = {}): void {
120    Analytics.track(eventName, params);
121  }
122  function flattenExtra(params: AnalyticsParams, extra?: Record<string, string | number | boolean>)...
123    if (!extra) return params;
124    return { ...params, ...extra };
125  }
126  function adParams(context: AnalyticsAdContext, extra: AnalyticsParams = {}): AnalyticsParams {
127    return flattenExtra({
128      ad_unit_id: context.adUnitId,
129      scene: context.scene,
130      ad_type: context.adType,
131      level_id: context.levelId == null ? null : String(context.levelId),
132      ...extra,
133    }, context.extra);
134  }
135  function errorToParams(error: unknown, extra: AnalyticsParams = {}): AnalyticsParams {
136    const err = error as { name?: string; message?: string; stack?: string; errMsg?: string; errCod...
137    return {
138      error_name: String(err?.name || 'Error'),
139      error_message: String(err?.message || err?.errMsg || error || 'unknown'),
140      error_code: err?.errorCode == null ? null : Number(err.errCode),
141      stack: err?.stack ? String(err.stack).slice(0, 500) : null,
142      ...extra,
143    };
144  }
145  export const analytics = {
146    track,
147    trackSessionStart(params: AnalyticsParams): void {
148      track(EVENT_NAMES.SESSION_START, params);
149    },
150    trackSessionEnd(reasonOrParams?: string | AnalyticsParams): void {
```

```
151   const params = typeof reasonOrParams === 'string'
152     ? { reason: reasonOrParams }
153     : (reasonOrParams || {});
154   track(EVENT_NAMES.SESSION_END, params);
155 },
156 trackAppShow(params: AnalyticsParams): void {
157   track('app_show', params);
158 },
159 trackAppError(error: unknown, params: AnalyticsParams = {}): void {
160   track(EVENT_NAMES.APP_ERROR, errorToParams(error, params));
161 },
162 trackLevelStart(event: LevelEvent): void {
163   track(EVENT_NAMES.LEVEL_START, flattenExtra({
164     level_id: String(event.levelId),
165     level_name: event.levelName || "",
166     mode: event.mode || 'level',
167   }, event.extra));
168 },
169 trackLevelClear(event: LevelEvent): void {
170   track(EVENT_NAMES.LEVEL_CLEAR, flattenExtra({
171     level_id: String(event.levelId),
172     level_name: event.levelName || "",
173     mode: event.mode || 'level',
174     duration_ms: event.durationMs ?? 0,
175   }, event.extra));
176 },
177 trackLevelFail(event: LevelEvent): void {
178   track(EVENT_NAMES.LEVEL_FAIL, flattenExtra({
179     level_id: String(event.levelId),
180     level_name: event.levelName || "",
181     mode: event.mode || 'level',
182     duration_ms: event.durationMs ?? 0,
183     reason: event.reason || 'unknown',
184   }, event.extra));
185 },
186 trackTutorialStep(event: TutorialEvent): void {
187   track('tutorial_step', flattenExtra({
188     step_id: event.stepId,
189     step_index: event.stepIndex ?? 0,
190     status: event.status || "",
191     is_force: !!event.isForce,
192   }, event.extra));
193 },
194 trackAdRequest(context: AnalyticsAdContext): void {
195   track(EVENT_NAMES.AD_REQUEST, adParams(context));
196 },
197 trackAdShow(context: AnalyticsAdContext): void {
198   track(EVENT_NAMES.AD_SHOW, adParams(context));
199 },
200 trackAdError(context: AnalyticsAdContext, error: { errCode?: number; errMsg?: string }): void {
```

```
201   track(EVENT_NAMES.AD_ERROR, adParams(context, {
202     err_code: error.errCode ?? -1,
203     err_msg: error.errMsg || 'unknown',
204   }));
205 },
206 trackAdClose(context: AnalyticsAdContext, params: AnalyticsParams = {}): void {
207   track(EVENT_NAMES.AD_CLOSE, adParams(context, params));
208 },
209 trackShareAppMessage(source: string, params: AnalyticsParams = {}): void {
210   track(EVENT_NAMES.SHARE_APP_MESSAGE, { source, channel: 'app_message', ...params });
211 },
212 trackShareTimeline(source: string, params: AnalyticsParams = {}): void {
213   track('share_timeline', { source, channel: 'timeline', ...params });
214 },
215 };
216 const APP_VERSION = '1.0.0';
217 const ENDPOINT = `${BACKEND_BASE_URL}/analytics-ingest/track`;
218 let initied = false;
219 export function initAnalytics(opts?: { endpoint?: string; userId?: string; debug?: boolean }): vo...
220   if (initied) return;
221   Analytics.init({
222     endpoint: opts?.endpoint || ENDPOINT,
223     gameKey: GAME_KEY,
224     appVersion: APP_VERSION,
225     platform: mapPlatform(),
226     deviceInfo: buildDeviceInfo(),
227     initialUserId: opts?.userId,
228     transport: { request: Platform.request.bind(Platform) },
229     storage: {
230       get: Platform.getStorageSync.bind(Platform),
231       set: Platform.setStorageSync.bind(Platform),
232       remove: Platform.removeStorageSync.bind(Platform),
233     },
234     lifecycle: { onHide: Platform.onHide.bind(Platform) },
235     debug: opts?.debug,
236   });
237   initied = true;
238 }
239 export function setAnalyticsUserId(userId: string): void {
240   if (!initied) return;
241   Analytics.setUserId(userId || '');
242 }
243 function mapPlatform(): PlatformName {
244   if (Platform.isWechat) return 'wechat';
245   if (Platform.isDouyin) return 'douyin';
246   if (Platform.isMinigame) return 'unknown';
247   return 'h5';
248 }
249 function buildDeviceInfo(): DeviceInfo {
250   const sys = Platform.getSystemInfoSync() || {};
```

```
251 return {
252   brand: String(sys.brand || sys.deviceBrand || ''),
253   model: String(sys.model || sys.deviceModel || ''),
254   system: String(sys.system || ''),
255   sdkVersion: String(sys.SDKVersion || sys.sdkVersion || ''),
256   screenWidth: Number(sys.screenWidth) || 0,
257   screenHeight: Number(sys.screenHeight) || 0,
258   network: 'unknown',
259 };
260 }
261 export const AUDIO_ASSETS = {
262   bgmClassic: 'subpkg_audio/audio/bgm_classic.mp3',
263   bgmLevel: 'subpkg_audio/audio/bgm_level.mp3',
264   eliminate: 'subpkg_audio/audio/sfx_eliminate.mp3',
265   eliminateBig: 'subpkg_audio/audio/sfx_eliminate_big.mp3',
266   moveLand: 'subpkg_audio/audio/sfx_move_land.mp3',
267   nextBalls: 'subpkg_audio/audio/sfx_next_balls.mp3',
268   button: 'subpkg_audio/audio/sfx_button.mp3',
269   propSuccess: 'subpkg_audio/audio/sfx_prop_success.mp3',
270   victory: 'subpkg_audio/audio/sfx_victory.mp3',
271 } as const;
272 export const AUDIO_VOLUME = {
273   bgmClassic: 0.42,
274   bgmLevel: 0.42,
275   eliminate: 0.78,
276   eliminateBig: 0.88,
277   moveLand: 0.7,
278   nextBalls: 0.68,
279   button: 0.72,
280   propSuccess: 0.82,
281   victory: 0.9,
282 } as const;
283 export const GAME_KEY = 'caizhu';
284 export const BACKEND_BASE_URL = 'https://rosa-env-d7grf78r5dbd37323.service.tcloudbase.com';
285 export const BACKEND_PATH_PREFIX = `${GAME_KEY}-api`;
286 export const BACKEND_LOGIN_PATH = `${BACKEND_PATH_PREFIX}/login/`;
287 export const BACKEND_PULL_PATH = `${BACKEND_PATH_PREFIX}/save/pull`;
288 export const BACKEND_PUSH_PATH = `${BACKEND_PATH_PREFIX}/save/push`;
289 export const BACKEND_HEALTH_PATH = `${BACKEND_PATH_PREFIX}/health`;
290 export const BACKEND_LB_CLASSIC_SUBMIT_PATH = `${BACKEND_PATH_PREFIX}/leaderboard/classic/submit`;
291 export const BACKEND_LB_LEVEL_SUBMIT_PATH = `${BACKEND_PATH_PREFIX}/leaderboard/level/submit`;
292 export const BACKEND_LB_CLASSIC_WORLD_PATH = `${BACKEND_PATH_PREFIX}/leaderboard/classic/world`;
293 export const BACKEND_LB_LEVEL_WORLD_PATH = `${BACKEND_PATH_PREFIX}/leaderboard/level/world`;
294 export const BACKEND_PROFILE_UPDATE_PATH = `${BACKEND_PATH_PREFIX}/profile/update`;
295 export const BACKEND_PROFILE_GET_PATH = `${BACKEND_PATH_PREFIX}/profile/get`;
296 export const BACKEND_REQUEST_TIMEOUT_MS = 10000;
297 export const BACKEND_TOKEN_KEY = `${GAME_KEY}_token`;
298 export const BACKEND_ANON_ID_KEY = `${GAME_KEY}_anon_id`;
299 export const LEVEL_PROGRESS_KEY = `${GAME_KEY}_level_progress`;
300 export const CLASSIC_RANKS_KEY = `${GAME_KEY}_classic_ranks`;
```

```
301 export const BEST_SCORE_KEY = `${GAME_KEY}_best_score`;
302 /** 经典模式进行中的局内存档（仅本地，不同步云端） */
303 export const CLASSIC_SAVED_GAME_KEY = `${GAME_KEY}_classic_saved_game`;
304 export const SKIN_STATE_KEY = `${GAME_KEY}_skin_state`;
305 export const PROP_INVENTORY_KEY = `${GAME_KEY}_prop_inventory`;
306 export const SEEN_SPECIAL_INTROS_KEY = `${GAME_KEY}_seen_special_piece_intros`;
307 export const USER_PROFILE_KEY = `${GAME_KEY}_user_profile`;
308 export const LEVEL1_TUTORIAL_KEY = `${GAME_KEY}_level1_tutorial_done`;
309 export const DEFAULT_AVATAR_PATH = 'subpkg_assets/images/avatar_default.png';
310 /** 微信好友榜云存储 KV 的 key（写入 wx.setUserCloudStorage，被开放数据域读取） */
311 export const WX_FRIEND_CLASSIC_KEY = 'classic_best';
312 export const WX_FRIEND_LEVEL_STARS_KEY = 'level_stars';
313 export const WX_FRIEND_LEVEL_SCORE_KEY = 'level_score';
314 export const CLOUD_SYNC_SCHEMA_VERSION = 1;
315 export const CLOUD_SYNC_META_KEY = `${GAME_KEY}_cloud_meta`;
316 export const CLOUD_SYNC_ALLOWLIST = [
317   LEVEL_PROGRESS_KEY,
318   CLASSIC_RANKS_KEY,
319   BEST_SCORE_KEY,
320   SKIN_STATE_KEY,
321   PROP_INVENTORY_KEY,
322   SEEN_SPECIAL_INTROS_KEY,
323   USER_PROFILE_KEY,
324 ] as const;
325 export const CLOUD_SYNC_EXCLUDE_KEYS = [
326   BACKEND_TOKEN_KEY,
327   BACKEND_ANON_ID_KEY,
328 ] as const;
329 export const CLOUD_SYNC_STARTUP_TIMEOUT_MS = 2500;
330 export const CLOUD_SYNC_DEBOUNCE_MS = 1500;
331 export const CLOUD_SYNC_BASE_DELAY_MS = 1500;
332 export const CLOUD_SYNC_MAX_BACKOFF_MS = 30000;
333 export const CLOUD_SYNC_MAX_FAIL_COUNT = 5;
334 export const CLOUD_SYNC_RETRY_INTERVAL_MS = 60000;
335 export const CLOUD_SYNC_LOG_THRESHOLD = 3;
336 export type CloudSyncKey = typeof CLOUD_SYNC_ALLOWLIST[number];
337 /** 游戏展示名（首页标题、日志、分享等） */
338 export const GAME_DISPLAY_NAME = '彩珠五连';
339 /** 新闻出版署《健康游戏忠告》全文（须在加载/启动画面显著位置登载） */
340 export const HEALTH_GAME_ADVISORY =
341   '抵制不良游戏，拒绝盗版游戏。\\n' +
342   '注意自我保护，谨防受骗上当。\\n' +
343   '适度游戏益脑，沉迷游戏伤身。\\n' +
344   '合理安排时间，享受健康生活。';
345 export const BOARD_SIZE = 9;
346 export const BOARD_CELLS = BOARD_SIZE * BOARD_SIZE; // 81
347 /**
348  * Ball color palette - optimized for maximum visual differentiation.
349  * Each entry: [main, highlight, shadow] for glass marble rendering.
350  */
```

```

351 export const BALL_PALETTE: readonly [main: number, hi: number, shadow: number][] = [
352   [0xDC3545, 0xFF6B7A, 0x8B0000], // red
353   [0xFFC107, 0xFFE066, 0xB8860B], // yellow
354   [0x0D6EFD, 0x6EA8FE, 0x003580], // blue
355   [0x198754, 0x5DD39E, 0x004D25], // green
356   [0x8B5CF6, 0xB794F6, 0x4C1D95], // purple
357   [0xFD7E14, 0xFFB066, 0xC05600], // orange
358   [0x0DCAF0, 0x6EDFF6, 0x087990], // cyan
359 ];
360 export const BALL_COLORS = BALL_PALETTE.map(p => p[0]);
361 export const COLOR_NAMES = ['红', '黄', '蓝', '绿', '紫', '橙', '青'] as const;
362 export const CLASSIC_COLOR_COUNT = 7;
363 export const MIN_LINE_LENGTH = 5;
364 export const BALLS_PER_TURN = 3;
365 export const INITIAL_BALLS = 5;
366 /** Score for eliminating exactly N balls in a line */
367 export function scoreForLine(count: number): number {
368   if (count < MIN_LINE_LENGTH) return 0;
369   if (count === 5) return 10;
370   if (count === 6) return 16;
371   if (count === 7) return 28;
372   if (count === 8) return 46;
373   return 70 + (count - 9) * 30;
374 }
375 export const CELL_GAP = 2;
376 /** Pushes score / preview / board / prop bar down to reduce empty space under the safe area */
377 export const PLAYFIELD_VERTICAL_OFFSET = 72;
378 export function computeBoardLayout(
379   logicWidth: number, logicHeight: number, safeTop: number,
380   opts?: { sidePadding?: number; maxCellSize?: number; bottomPadding?: number },
381 ) {
382   const topBarHeight = 120;
383   const previewHeight = 60;
384   const bottomPadding = opts?.bottomPadding ?? 200;
385   const sidePadding = opts?.sidePadding ?? 20;
386   const cellSizeCap = opts?.maxCellSize ?? 72;
387   const availableWidth = logicWidth - sidePadding * 2;
388   const availableHeight =
389     logicHeight - safeTop - topBarHeight - previewHeight - bottomPadding - PLAYFIELD_VERTICAL_OFF...
390   const maxCellSize = Math.min(
391     Math.floor((availableWidth - CELL_GAP * (BOARD_SIZE - 1)) / BOARD_SIZE),
392     Math.floor((availableHeight - CELL_GAP * (BOARD_SIZE - 1)) / BOARD_SIZE)
393   );
394   const cellSize = Math.min(maxCellSize, cellSizeCap);
395   const boardPixelSize = cellSize * BOARD_SIZE + CELL_GAP * (BOARD_SIZE - 1);
396   const boardX = Math.floor((logicWidth - boardPixelSize) / 2);
397   const boardY = safeTop + topBarHeight + previewHeight + PLAYFIELD_VERTICAL_OFFSET;
398   return {
399     cellSize,
400     boardPixelSize,

```



```
401   boardX,
402   boardY,
403   topBarY: safeTop + PLAYFIELD_VERTICAL_OFFSET,
404   previewY: safeTop + topBarHeight + PLAYFIELD_VERTICAL_OFFSET,
405   };
406 }
407 import { createNormalPiece, type Piece } from '@config/PieceConfig';
408 import type { Point } from '@systems/PathFinder';
409 export interface Level1TutorialCell {
410   row: number;
411   col: number;
412   piece: Piece;
413 }
414 export interface Level1TutorialLayout {
415   initialCells: readonly Level1TutorialCell[];
416   nextPieces: readonly [Piece, Piece, Piece];
417   nextLandingPositions: readonly [Point, Point, Point];
418   source: Point;
419   target: Point;
420 }
421 /**
422  * 第 1 关教程棋盘:
423  * - 红珠五连教学: 从 (6,6) 移到 (4,6) 完成横排 4 列 26
424  * - 盘面另有黄/蓝珠, 消除五连后仍留在棋盘上 (消除不落子规则)
425  */
426 export const LEVEL1_TUTORIAL_LAYOUT: Level1TutorialLayout = {
427   initialCells: [
428     { row: 4, col: 2, piece: createNormalPiece(0) },
429     { row: 4, col: 3, piece: createNormalPiece(0) },
430     { row: 4, col: 4, piece: createNormalPiece(0) },
431     { row: 4, col: 5, piece: createNormalPiece(0) },
432     { row: 6, col: 6, piece: createNormalPiece(0) },
433     { row: 2, col: 1, piece: createNormalPiece(1) },
434     { row: 2, col: 7, piece: createNormalPiece(2) },
435     { row: 3, col: 3, piece: createNormalPiece(2) },
436     { row: 3, col: 6, piece: createNormalPiece(1) },
437     { row: 5, col: 1, piece: createNormalPiece(1) },
438     { row: 5, col: 7, piece: createNormalPiece(2) },
439     { row: 6, col: 2, piece: createNormalPiece(2) },
440     { row: 6, col: 0, piece: createNormalPiece(1) },
441     { row: 7, col: 4, piece: createNormalPiece(1) },
442     { row: 1, col: 5, piece: createNormalPiece(2) },
443     { row: 7, col: 7, piece: createNormalPiece(2) },
444   ],
445   nextPieces: [
446     createNormalPiece(1),
447     createNormalPiece(2),
448     createNormalPiece(1),
449   ],
450   nextLandingPositions: [
```

```
451   { row: 0, col: 0 },
452   { row: 0, col: 1 },
453   { row: 0, col: 2 },
454 ],
455 source: { row: 6, col: 6 },
456 target: { row: 4, col: 6 },
457 };
458 import type { PieceSpawnKind } from '@config/PieceConfig';
459 export interface LevelDef {
460   id: number;
461   type: 'steps' | 'timed';
462   colorCount: number;
463   starScores: readonly [oneStar: number, twoStar: number, threeStar: number];
464   stepLimit?: number;
465   timeLimit?: number;
466   initialBalls: number;
467   ballsPerTurn: number;
468   noSpawnThreshold: number;
469   /** Chance (0-1) of spawning a wild/rainbow ball instead of a normal color */
470   wildBallChance?: number;
471   /** Chance (0-1) of spawning a bomb ball */
472   bombBallChance?: number;
473   /** Chance (0-1) of spawning a frozen ball that cannot be moved */
474   frozenBallChance?: number;
475   /** Chance (0-1) of spawning a chained ball */
476   chainBallChance?: number;
477   /** Chance (0-1) of spawning a board blocker */
478   blockChance?: number;
479   /** Special pieces guaranteed in the initial board setup */
480   guaranteedInitialPieces?: readonly PieceSpawnKind[];
481   /** Special pieces guaranteed in the first preview/spawn queue */
482   guaranteedNextPieces?: readonly PieceSpawnKind[];
483   /** Designed board layout used instead of fully random initial placement */
484   layoutId?: string;
485 }
486 function stars(oneStar: number): readonly [number, number, number] {
487   return [oneStar, Math.round(oneStar * 2.4), Math.round(oneStar * 4.5)] as const;
488 }
489 export function getPassScore(starScores: readonly [number, number, number]): number {
490   return starScores[0];
491 }
492 export function getMaxStarScore(starScores: readonly [number, number, number]): number {
493   return starScores[2];
494 }
495 export function getLevelStars(score: number, starScores: readonly [number, number, number]): numb...
496   if (score >= starScores[2]) return 3;
497   if (score >= starScores[1]) return 2;
498   if (score >= starScores[0]) return 1;
499   return 0;
500 }
```

```

501 export const LEVELS: readonly LevelDef[] = [
502   // === Tier 1: 入门 (1-5) - unlock all colors early ===
503   { id: 1, type: 'steps', colorCount: 3, starScores: stars(10), stepLimit: 80, initialBalls: 16...
504   { id: 2, type: 'steps', colorCount: 4, starScores: stars(30), stepLimit: 76, initialBalls: 8,...
505   { id: 3, type: 'steps', colorCount: 5, starScores: stars(50), stepLimit: 76, initialBalls: 9,...
506   { id: 4, type: 'steps', colorCount: 6, starScores: stars(75), stepLimit: 72, initialBalls: 10...
507   { id: 5, type: 'steps', colorCount: 7, starScores: stars(100), stepLimit: 72, initialBalls: 11...
508   // === Tier 2: 上手 (6-10) - introduce one special mechanic per level ===
509   { id: 6, type: 'steps', colorCount: 7, starScores: stars(90), stepLimit: 68, initialBalls: 8,...
510   { id: 7, type: 'steps', colorCount: 7, starScores: stars(110), stepLimit: 68, initialBalls: 8,...
511   { id: 8, type: 'steps', colorCount: 7, starScores: stars(130), stepLimit: 68, initialBalls: 8,...
512   { id: 9, type: 'steps', colorCount: 7, starScores: stars(150), stepLimit: 68, initialBalls: 8,...
513   { id: 10, type: 'steps', colorCount: 7, starScores: stars(180), stepLimit: 72, initialBalls: 10...
514   // === Tier 3: 进阶 (11-15) - combine low-density specials ===
515   { id: 11, type: 'timed', colorCount: 7, starScores: stars(220), timeLimit: 360, initialBalls: 9...
516   { id: 12, type: 'timed', colorCount: 7, starScores: stars(250), timeLimit: 360, initialBalls: 9...
517   { id: 13, type: 'steps', colorCount: 7, starScores: stars(260), stepLimit: 72, initialBalls: 10...
518   { id: 14, type: 'timed', colorCount: 7, starScores: stars(300), timeLimit: 420, initialBalls: 1...
519   { id: 15, type: 'steps', colorCount: 7, starScores: stars(330), stepLimit: 80, initialBalls: 14...
520   // === Tier 4: 挑战 (16-20) - denser starts with low-density specials ===
521   { id: 16, type: 'steps', colorCount: 7, starScores: stars(360), stepLimit: 84, initialBalls: 12...
522   { id: 17, type: 'steps', colorCount: 7, starScores: stars(390), stepLimit: 84, initialBalls: 10...
523   { id: 18, type: 'steps', colorCount: 7, starScores: stars(420), stepLimit: 88, initialBalls: 12...
524   { id: 19, type: 'timed', colorCount: 7, starScores: stars(480), timeLimit: 480, initialBalls: 1...
525   { id: 20, type: 'timed', colorCount: 7, starScores: stars(520), timeLimit: 480, initialBalls: 1...
526   // === Tier 5: 困难 (21-25) - mixed mechanics with stronger initial boards ===
527   { id: 21, type: 'steps', colorCount: 7, starScores: stars(560), stepLimit: 92, initialBalls: 14...
528   { id: 22, type: 'steps', colorCount: 7, starScores: stars(600), stepLimit: 96, initialBalls: 14...
529   { id: 23, type: 'steps', colorCount: 7, starScores: stars(640), stepLimit: 96, initialBalls: 13...
530   { id: 24, type: 'timed', colorCount: 7, starScores: stars(700), timeLimit: 540, initialBalls: 1...
531   { id: 25, type: 'timed', colorCount: 7, starScores: stars(760), timeLimit: 600, initialBalls: 1...
532   // === Tier 6: 大师 (26-30) - chain balls and blockers ===
533   { id: 26, type: 'steps', colorCount: 7, starScores: stars(780), stepLimit: 104, initialBalls: ...
534   { id: 27, type: 'steps', colorCount: 7, starScores: stars(850), stepLimit: 108, initialBalls: ...
535   { id: 28, type: 'steps', colorCount: 7, starScores: stars(900), stepLimit: 112, initialBalls: ...
536   { id: 29, type: 'timed', colorCount: 7, starScores: stars(950), timeLimit: 600, initialBalls: ...
537   { id: 30, type: 'timed', colorCount: 7, starScores: stars(1000), timeLimit: 600, initialBalls: ...
538 ];
539 export function getLevelDef(id: number): LevelDef | undefined {
540   return LEVELS.find(l => l.id === id);
541 }
542 export const TOTAL_LEVELS = LEVELS.length;
543 import {
544   createNormalPiece,
545   type Piece,
546 } from '@config/PieceConfig';
547 import type { Point } from '@systems/PathFinder';
548 export interface DesignedLevelLayout {
549   initialCells: readonly { row: number; col: number; piece: Piece }[];
550   nextPieces: readonly Piece[];

```

```

551   nextLandingPositions?: readonly Point[];
552 }
553 const n = (color: number): Piece => createNormalPiece(color);
554 const wild = (): Piece => ({ kind: 'wild' });
555 const bomb = (color: number): Piece => ({ kind: 'bomb', color });
556 const frozen = (color: number): Piece => ({ kind: 'frozen', color, layers: 1 });
557 const chain = (color: number): Piece => ({ kind: 'chain', color, layers: 1 });
558 const block = (): Piece => ({ kind: 'block', layers: 1 });
559 function cell(row: number, col: number, piece: Piece): { row: number; col: number; piece: Piece } {
560   return { row, col, piece };
561 }
562 function layout(
563   initialCells: readonly { row: number; col: number; piece: Piece }[],
564   nextPieces: readonly Piece[],
565   nextLandingPositions?: readonly Point[],
566 ): DesignedLevelLayout {
567   return { initialCells, nextPieces, nextLandingPositions };
568 }
569 export const LEVEL_LAYOUTS: Record<string, DesignedLevelLayout> = {
570   level_01: layout([
571     cell(4, 2, n(0)), cell(4, 3, n(0)), cell(4, 4, n(0)), cell(4, 5, n(0)),
572     cell(6, 6, n(0)),
573     cell(2, 1, n(1)), cell(2, 7, n(2)), cell(3, 3, n(2)), cell(3, 6, n(1)),
574     cell(5, 1, n(1)), cell(5, 7, n(2)), cell(6, 2, n(2)), cell(6, 0, n(1)),
575     cell(7, 4, n(1)), cell(1, 5, n(2)), cell(7, 7, n(2)),
576   ], [n(1), n(2), n(1)]),
577   level_02: layout([
578     cell(2, 2, n(0)), cell(2, 3, n(0)), cell(2, 4, n(0)), cell(4, 6, n(0)),
579     cell(6, 1, n(1)), cell(6, 2, n(1)), cell(6, 4, n(1)), cell(7, 7, n(2)),
580   ], [n(0), n(1), n(2)]),
581   level_03: layout([
582     cell(1, 1, n(2)), cell(1, 2, n(2)), cell(1, 3, n(2)), cell(1, 5, n(2)),
583     cell(5, 2, n(0)), cell(5, 3, n(0)), cell(5, 5, n(0)), cell(6, 5, n(0)),
584     cell(7, 7, n(3)),
585   ], [n(2), n(0), n(4)]),
586   level_04: layout([
587     cell(3, 1, n(1)), cell(3, 2, n(1)), cell(3, 4, n(1)), cell(3, 5, n(1)),
588     cell(1, 4, n(2)), cell(2, 4, n(2)), cell(4, 4, n(2)), cell(5, 4, n(2)),
589     cell(7, 1, n(3)), cell(7, 7, n(4)),
590   ], [n(1), n(2), n(5)]),
591   level_05: layout([
592     cell(2, 1, n(3)), cell(2, 2, n(3)), cell(2, 3, n(3)), cell(2, 5, n(3)),
593     cell(5, 5, n(4)), cell(6, 5, n(4)), cell(7, 5, n(4)), cell(4, 5, n(4)),
594     cell(6, 1, n(0)), cell(6, 2, n(0)), cell(6, 4, n(0)),
595   ], [n(3), n(4), n(0)]),
596   level_06: layout([
597     cell(4, 1, n(0)), cell(4, 2, n(0)), cell(4, 3, n(0)), cell(4, 5, n(0)),
598     cell(6, 3, wild()), cell(2, 6, n(1)), cell(3, 6, n(1)), cell(5, 6, n(1)),
599   ], [wild(), n(0), n(1)]),
600   level_07: layout([

```

```
601     cell(3, 1, n(2)), cell(3, 2, n(2)), cell(3, 3, bomb(2)), cell(3, 5, n(2)),
602     cell(6, 1, n(4)), cell(6, 2, n(4)), cell(6, 4, n(4)), cell(7, 7, n(0)),
603 ], [n(2), n(4), n(5)]),
604 level_08: layout([
605     cell(4, 1, n(1)), cell(4, 2, n(1)), cell(4, 3, frozen(1)), cell(4, 5, n(1)),
606     cell(2, 5, n(1)), cell(6, 2, n(3)), cell(6, 3, n(3)), cell(6, 5, n(3)),
607 ], [n(1), n(3), n(1)]),
608 level_09: layout([
609     cell(2, 2, n(5)), cell(2, 3, n(5)), cell(2, 4, chain(5)), cell(2, 6, n(5)),
610     cell(5, 1, n(0)), cell(5, 2, n(0)), cell(5, 4, n(0)), cell(7, 5, n(2)),
611 ], [n(5), n(0), n(5)]),
612 level_10: layout([
613     cell(3, 3, block()), cell(3, 4, block()), cell(4, 3, block()),
614     cell(4, 4, bomb(0)), cell(4, 5, n(0)), cell(4, 6, n(0)), cell(4, 7, n(0)),
615     cell(6, 2, n(1)), cell(6, 3, n(1)), cell(6, 5, n(1)),
616 ], [n(0), n(1), bomb(0)]),
617 level_11: layout([
618     cell(2, 1, n(2)), cell(2, 2, n(2)), cell(2, 3, bomb(2)), cell(2, 5, n(2)),
619     cell(5, 3, wild()), cell(6, 3, n(4)), cell(6, 4, n(4)), cell(6, 6, n(4)),
620     cell(7, 1, block()),
621 ], [wild(), n(2), n(4)]),
622 level_12: layout([
623     cell(1, 2, n(3)), cell(2, 2, frozen(3)), cell(3, 2, n(3)), cell(5, 2, n(3)),
624     cell(5, 5, n(1)), cell(5, 6, frozen(1)), cell(5, 7, n(1)), cell(7, 7, n(1)),
625     cell(3, 6, bomb(3)),
626 ], [n(3), n(1), wild()]),
627 level_13: layout([
628     cell(3, 1, n(4)), cell(3, 2, chain(4)), cell(3, 3, bomb(4)), cell(3, 5, n(4)),
629     cell(1, 6, n(0)), cell(2, 6, n(0)), cell(4, 6, chain(0)), cell(5, 6, n(0)),
630     cell(7, 2, block()), cell(7, 3, block()),
631 ], [n(4), n(0), bomb(4)]),
632 level_14: layout([
633     cell(3, 3, block()), cell(3, 4, block()), cell(4, 3, block()), cell(5, 3, block()),
634     cell(2, 2, n(2)), cell(2, 3, n(2)), cell(2, 5, n(2)), cell(2, 6, n(2)),
635     cell(5, 5, bomb(2)), cell(6, 6, n(5)), cell(6, 7, n(5)), cell(7, 6, n(5)),
636 ], [n(2), n(5), wild()]),
637 level_15: layout([
638     cell(1, 1, n(0)), cell(1, 2, n(0)), cell(1, 4, frozen(0)), cell(1, 5, n(0)),
639     cell(3, 2, chain(3)), cell(3, 3, n(3)), cell(3, 4, n(3)), cell(3, 6, n(3)),
640     cell(5, 4, block()), cell(5, 5, block()), cell(6, 5, bomb(3)),
641     cell(7, 1, n(5)), cell(7, 2, n(5)), cell(7, 4, n(5)),
642 ], [n(0), n(3), n(5)]),
643 level_16: layout([
644     cell(2, 4, block()), cell(3, 4, block()), cell(4, 4, block()), cell(5, 4, block()),
645     cell(2, 1, n(1)), cell(2, 2, n(1)), cell(2, 3, n(1)), cell(2, 6, n(1)),
646     cell(6, 5, bomb(1)), cell(6, 6, n(4)), cell(6, 7, n(4)), cell(7, 6, n(4)),
647 ], [n(1), n(4), wild()]),
648 level_17: layout([
649     cell(3, 1, n(2)), cell(3, 2, n(2)), cell(3, 3, bomb(2)), cell(3, 5, n(2)),
650     cell(4, 5, bomb(5)), cell(5, 5, n(5)), cell(6, 5, n(5)), cell(7, 5, n(5)),
```

```
651     cell(2, 6, block()), cell(5, 2, chain(2)),
652 ], [n(2), n(5), bomb(2)]),
653 level_18: layout([
654     cell(1, 1, n(0)), cell(1, 2, frozen(0)), cell(1, 3, n(0)), cell(1, 5, chain(0)), cell(1, 6, n...
655     cell(4, 2, n(6)), cell(4, 3, chain(6)), cell(4, 4, frozen(6)), cell(4, 6, n(6)),
656     cell(6, 6, bomb(0)), cell(7, 2, block()), cell(7, 3, block()),
657 ], [n(0), n(6), wild()]),
658 level_19: layout([
659     cell(1, 1, n(1)), cell(1, 2, n(1)), cell(2, 1, n(1)), cell(2, 2, n(1)),
660     cell(1, 7, n(3)), cell(2, 7, n(3)), cell(2, 6, n(3)),
661     cell(6, 1, n(5)), cell(7, 1, n(5)), cell(7, 2, n(5)),
662     cell(6, 7, n(0)), cell(7, 7, n(0)), cell(7, 6, n(0)),
663     cell(4, 4, block()), cell(4, 5, bomb(1)),
664 ], [n(1), n(3), n(5)]),
665 level_20: layout([
666     cell(3, 3, block()), cell(3, 4, block()), cell(3, 5, block()),
667     cell(4, 3, block()), cell(4, 5, block()), cell(5, 3, block()), cell(5, 4, block()),
668     cell(4, 4, bomb(2)), cell(2, 4, n(2)), cell(6, 4, n(2)), cell(4, 2, n(2)), cell(4, 6, n(2)),
669     cell(1, 1, n(4)), cell(1, 2, n(4)), cell(1, 4, n(4)),
670 ], [n(2), n(4), bomb(2)]),
671 level_21: layout([
672     cell(2, 1, n(0)), cell(2, 2, n(0)), cell(2, 4, n(0)), cell(2, 6, n(0)),
673     cell(4, 2, n(2)), cell(4, 3, n(2)), cell(4, 5, n(2)), cell(4, 7, n(2)),
674     cell(6, 1, n(3)), cell(6, 3, n(3)), cell(6, 4, n(3)), cell(6, 6, n(3)),
675     cell(3, 6, frozen(2)), cell(7, 7, chain(3)),
676 ], [n(0), n(2), n(3)]),
677 level_22: layout([
678     cell(2, 3, block()), cell(2, 4, block()), cell(3, 3, block()), cell(4, 3, block()),
679     cell(4, 4, bomb(5)), cell(4, 5, n(5)), cell(4, 6, n(5)), cell(4, 7, n(5)),
680     cell(6, 2, n(1)), cell(6, 3, n(1)), cell(6, 5, n(1)), cell(7, 5, n(1)),
681     cell(1, 6, chain(5)), cell(7, 1, frozen(1)),
682 ], [n(5), n(1), bomb(5)]),
683 level_23: layout([
684     cell(1, 1, n(4)), cell(1, 2, frozen(4)), cell(1, 3, frozen(4)), cell(1, 5, n(4)), cell(1, 6, ...
685     cell(4, 2, n(0)), cell(4, 3, frozen(0)), cell(4, 4, frozen(0)), cell(4, 6, n(0)),
686     cell(6, 5, bomb(4)), cell(7, 6, chain(0)), cell(3, 7, block()), cell(4, 7, block()),
687 ], [n(4), n(0), wild()]),
688 level_24: layout([
689     cell(1, 1, n(1)), cell(1, 2, n(1)), cell(2, 1, n(1)), cell(2, 3, n(1)),
690     cell(1, 7, n(2)), cell(2, 7, n(2)), cell(3, 7, n(2)), cell(2, 5, n(2)),
691     cell(6, 1, n(3)), cell(7, 1, n(3)), cell(7, 2, n(3)), cell(5, 2, n(3)),
692     cell(6, 7, n(5)), cell(7, 7, n(5)), cell(7, 6, n(5)), cell(5, 6, n(5)),
693     cell(4, 4, bomb(1)), cell(4, 5, block()), cell(3, 4, chain(2)),
694 ], [n(1), n(2), n(3)]),
695 level_25: layout([
696     cell(1, 1, bomb(0)), cell(1, 2, n(0)), cell(1, 3, n(0)), cell(1, 5, n(0)),
697     cell(3, 1, frozen(2)), cell(3, 2, n(2)), cell(3, 3, frozen(2)), cell(3, 5, n(2)),
698     cell(5, 1, chain(4)), cell(5, 2, n(4)), cell(5, 3, chain(4)), cell(5, 5, n(4)),
699     cell(6, 6, block()), cell(6, 7, block()), cell(7, 6, bomb(6)), cell(7, 7, n(6)),
700     cell(4, 4, wild()),
```

```

701 ], [n(0), n(2), n(4)]),
702 level_26: layout([
703   cell(1, 1, n(0)), cell(1, 2, n(0)), cell(1, 4, n(0)), cell(1, 6, n(0)),
704   cell(3, 1, n(2)), cell(3, 2, n(2)), cell(3, 4, n(2)), cell(3, 6, n(2)),
705   cell(5, 1, n(4)), cell(5, 3, n(4)), cell(5, 4, n(4)), cell(5, 6, n(4)),
706   cell(7, 2, n(6)), cell(7, 3, n(6)), cell(7, 5, n(6)), cell(7, 7, n(6)),
707   cell(4, 4, wild()), cell(2, 7, chain(0)),
708 ], [n(0), n(2), n(4)]),
709 level_27: layout([
710   cell(2, 2, bomb(1)), cell(2, 3, n(1)), cell(2, 4, n(1)), cell(2, 6, n(1)),
711   cell(6, 6, bomb(5)), cell(6, 5, n(5)), cell(6, 4, n(5)), cell(6, 2, n(5)),
712   cell(3, 4, block()), cell(4, 4, block()), cell(5, 4, block()),
713   cell(4, 2, n(3)), cell(4, 3, n(3)), cell(4, 5, n(3)), cell(4, 6, n(3)),
714   cell(1, 7, frozen(1)), cell(7, 1, chain(5)),
715 ], [n(1), n(5), n(3)]),
716 level_28: layout([
717   cell(2, 1, n(2)), cell(2, 2, frozen(2)), cell(2, 3, chain(2)), cell(2, 4, frozen(2)), cell(2,...
718   cell(4, 1, n(0)), cell(4, 2, chain(0)), cell(4, 3, frozen(0)), cell(4, 5, chain(0)), cell(4, ...
719   cell(6, 2, n(5)), cell(6, 3, frozen(5)), cell(6, 4, chain(5)), cell(6, 6, n(5)),
720   cell(3, 7, bomb(2)), cell(5, 7, block()), cell(7, 7, block()),
721 ], [n(2), n(0), n(5)]),
722 level_29: layout([
723   cell(1, 1, n(0)), cell(1, 2, n(0)), cell(1, 3, n(0)), cell(1, 5, n(0)),
724   cell(2, 6, bomb(0)), cell(3, 6, n(3)), cell(4, 6, n(3)), cell(5, 6, n(3)),
725   cell(6, 1, n(4)), cell(6, 2, n(4)), cell(6, 4, n(4)), cell(7, 4, n(4)),
726   cell(3, 2, block()), cell(3, 3, block()), cell(4, 2, frozen(4)), cell(4, 3, chain(3)),
727   cell(7, 7, wild()), cell(5, 1, bomb(4)),
728 ], [n(0), n(3), n(4)]),
729 level_30: layout([
730   cell(3, 3, block()), cell(3, 4, block()), cell(3, 5, block()),
731   cell(4, 3, block()), cell(4, 4, bomb(2)), cell(4, 5, block()),
732   cell(5, 3, block()), cell(5, 4, block()), cell(5, 5, block()),
733   cell(1, 1, n(0)), cell(1, 2, n(0)), cell(1, 4, frozen(0)), cell(1, 6, n(0)),
734   cell(7, 1, n(3)), cell(7, 2, chain(3)), cell(7, 4, n(3)), cell(7, 6, n(3)),
735   cell(1, 7, bomb(5)), cell(2, 7, n(5)), cell(3, 7, n(5)), cell(5, 7, n(5)),
736   cell(6, 1, wild()), cell(6, 6, bomb(3)),
737 ], [n(2), n(0), n(3)]),
738 };
739 export function getLevelLayout(layoutId?: string): DesignedLevelLayout | undefined {
740   return layoutId ? LEVEL_LAYOUTS[layoutId] : undefined;
741 }
742 import { BALL_COLORS } from '@config/GameConfig';
743 export type PieceKind = 'normal' | 'wild' | 'bomb' | 'frozen' | 'chain' | 'block';
744 export interface NormalPiece {
745   kind: 'normal';
746   color: number;
747 }
748 export interface WildPiece {
749   kind: 'wild';
750 }

```

```
751 export interface BombPiece {
752   kind: 'bomb';
753   color: number;
754 }
755 export interface FrozenPiece {
756   kind: 'frozen';
757   color: number;
758   layers: 1;
759 }
760 export interface ChainPiece {
761   kind: 'chain';
762   color: number;
763   layers: 1 | 2;
764 }
765 export interface BlockPiece {
766   kind: 'block';
767   layers: 1 | 2;
768 }
769 export type Piece = NormalPiece | WildPiece | BombPiece | FrozenPiece | ChainPiece | BlockPiece;
770 export type CellValue = Piece | null;
771 export interface SpecialPieceChances {
772   wildBallChance?: number;
773   bombBallChance?: number;
774   frozenBallChance?: number;
775   chainBallChance?: number;
776   blockChance?: number;
777 }
778 export type PieceSpawnKind = 'wild' | 'bomb' | 'frozen' | 'chain' | 'block';
779 export function createNormalPiece(color: number): NormalPiece {
780   return { kind: 'normal', color };
781 }
782 export function clonePiece(piece: Piece): Piece {
783   return { ...piece } as Piece;
784 }
785 export function cloneCell(cell: CellValue): CellValue {
786   return cell ? clonePiece(cell) : null;
787 }
788 export function getPieceColor(piece: Piece | null): number | null {
789   if (!piece) return null;
790   switch (piece.kind) {
791     case 'normal':
792     case 'bomb':
793     case 'frozen':
794     case 'chain':
795     return piece.color;
796   default:
797     return null;
798 }
799 }
800 export function getPieceDisplayColor(piece: Piece | null): number {
```



```
801 if (!piece) return 0xCCCCCC;
802 if (piece.kind === 'wild') return 0xFFFFF;
803 if (piece.kind === 'bomb') return 0xF39C12;
804 if (piece.kind === 'block') return 0x8EA0B8;
805 const color = getPieceColor(piece);
806 return color === null ? 0xCCCCCC : (BALL_COLORS[color] ?? 0xCCCCCC);
807 }
808 export function isMovablePiece(piece: Piece | null): boolean {
809   return !!piece && piece.kind !== 'frozen' && piece.kind !== 'block';
810 }
811 export function isLineParticipant(piece: Piece | null): boolean {
812   return !!piece && piece.kind !== 'block';
813 }
814 export function pieceMatchesColor(piece: Piece | null, color: number): boolean {
815   if (!isLineParticipant(piece)) return false;
816   if (piece.kind === 'wild') return true;
817   return getPieceColor(piece) === color;
818 }
819 export function isWildPiece(piece: Piece | null): boolean {
820   return piece?.kind === 'wild';
821 }
822 export function isBombPiece(piece: Piece | null): boolean {
823   return piece?.kind === 'bomb';
824 }
825 export function isBlockPiece(piece: Piece | null): boolean {
826   return piece?.kind === 'block';
827 }
828 export function isDamagedByLine(piece: Piece): boolean {
829   return piece.kind !== 'block';
830 }
831 export function damagePieceByLine(piece: Piece): CellValue {
832   if (piece.kind === 'frozen') return createNormalPiece(piece.color);
833   if (piece.kind === 'chain') {
834     if (piece.layers > 1) return { ...piece, layers: 1 };
835     return createNormalPiece(piece.color);
836   }
837   return null;
838 }
839 export function damagePieceByExplosion(piece: Piece): CellValue {
840   if (piece.kind === 'block') {
841     if (piece.layers > 1) return { ...piece, layers: 1 };
842     return null;
843   }
844   return damagePieceByLine(piece);
845 }
846 export function rollSpawnPiece(colorCount: number, chances: SpecialPieceChances = {}): Piece {
847   const color = Math.floor(Math.random() * colorCount);
848   let cursor = 0;
849   const rand = Math.random();
850   cursor += chances.wildBallChance ?? 0;
```

```
851   if (rand < cursor) return { kind: 'wild' };
852   cursor += chances.bombBallChance ?? 0;
853   if (rand < cursor) return { kind: 'bomb', color };
854   cursor += chances.frozenBallChance ?? 0;
855   if (rand < cursor) return { kind: 'frozen', color, layers: 1 };
856   cursor += chances.chainBallChance ?? 0;
857   if (rand < cursor) return { kind: 'chain', color, layers: 1 };
858   cursor += chances.blockChance ?? 0;
859   if (rand < cursor) return { kind: 'block', layers: 1 };
860   return createNormalPiece(color);
861 }
862 export function createSpecialPiece(kind: PieceSpawnKind, colorCount: number): Piece {
863   const color = Math.floor(Math.random() * colorCount);
864   switch (kind) {
865     case 'wild':
866       return { kind: 'wild' };
867     case 'bomb':
868       return { kind: 'bomb', color };
869     case 'frozen':
870       return { kind: 'frozen', color, layers: 1 };
871     case 'chain':
872       return { kind: 'chain', color, layers: 1 };
873     case 'block':
874       return { kind: 'block', layers: 1 };
875   }
876 }
877 /** Prop types available in-game */
878 export enum PropType {
879   ColorBlast = 'colorBlast',
880   CrossClear = 'crossClear',
881   WildNext = 'wildNext',
882 }
883 export const PROP_REWARDED_AD_UNIT_ID = 'adunit-fab969e9ff8cd7d7';
884 export interface PropDef {
885   type: PropType;
886   name: string;
887   icon: string;
888   description: string;
889   /** Max uses per game session */
890   maxPerGame: number;
891   /** Ad unit ID placeholder - fill with real ID when monetizing */
892   adUnitId: string;
893 }
894 export const PROP_DEFS: Record<PropType, PropDef> = {
895   [PropType.ColorBlast]: {
896     type: PropType.ColorBlast,
897     name: '同色爆破',
898     icon: '?',
899     description: '随机消除棋盘上一种颜色的全部珠子',
900     maxPerGame: 1,
```

```
901   adUnitId: PROP_REWARDED_AD_UNIT_ID,
902 },
903 [PropType.CrossClear]: {
904   type: PropType.CrossClear,
905   name: '十字清场',
906   icon: "",
907   description: '点击一个格子，消除所在行和列的珠子',
908   maxPerGame: 1,
909   adUnitId: PROP_REWARDED_AD_UNIT_ID,
910 },
911 [PropType.WildNext]: {
912   type: PropType.WildNext,
913   name: '万能预备',
914   icon: '?',
915   description: '把下一轮出现的珠子全部变成万能珠',
916   maxPerGame: 1,
917   adUnitId: PROP_REWARDED_AD_UNIT_ID,
918 },
919 };
920 export const ALL_PROPS = Object.values(PROP_DEFS);
921 import { GAME_DISPLAY_NAME } from './GameConfig';
922 export const SHARE_IMAGES = {
923   appMessage: 'subpkg_assets/images/share_card_combo_yellow.jpg',
924   timeline: 'subpkg_assets/images/share_card_board_ocean.jpg',
925   /** 经典模式破纪录炫耀分享 */
926   classicRecord: 'subpkg_assets/images/share_card_classic_record.jpg',
927 } as const;
928 export const SHARE_TITLES = {
929   appMessage: `来玩${GAME_DISPLAY_NAME}，五颗连成线！`,
930   timeline: `${GAME_DISPLAY_NAME}，越玩越上头`,
931 } as const;
932 export function buildShareQuery(source: string): string {
933   return `from=share&source=${encodeURIComponent(source)}`;
934 }
935 /** 破纪录分享文案（title 可带动态分数，炫耀+挑战好友） */
936 export function buildClassicRecordShareTitle(score: number): string {
937   return `经典模式 ${score} 分新纪录！不服来比比？`;
938 }
939 export type SkinCategory = 'orb' | 'background';
940 export type UnlockCondition =
941   | { type: 'default' }
942   | { type: 'classicScore'; score: number }
943   | { type: 'levelReached'; level: number }
944   | { type: 'ad'; adUnitId: string }
945   | { type: 'future'; label: string };
946 export interface BaseSkinDef {
947   id: string;
948   category: SkinCategory;
949   name: string;
950   description: string;
```

```
951  unlock: UnlockCondition;
952  }
953  export interface OrbSkinDef extends BaseSkinDef {
954    category: 'orb';
955    sheetRow: number;
956  }
957  export interface BackgroundSkinDef extends BaseSkinDef {
958    category: 'background';
959    imagePath: string;
960    previewPath: string;
961  }
962  export type SkinDef = OrbSkinDef | BackgroundSkinDef;
963  export const SKIN_REWARDED_AD_UNIT_ID = 'adunit-d75361ec8061cdf';
964  export const DEFAULT_ORB_SKIN_ID = 'glass';
965  export const DEFAULT_BACKGROUND_SKIN_ID = 'default-bg';
966  export const DEFAULT_CLASSIC_BACKGROUND = 'subpkg_assets/images/bg_classic_redesign.png';
967  export const DEFAULT_LEVEL_BACKGROUND = 'subpkg_assets/images/bg_level_redesign.png';
968  export const ORB_SKINS: readonly OrbSkinDef[] = [
969    {
970      id: 'glass',
971      category: 'orb',
972      name: '经典玻璃',
973      description: '默认彩珠皮肤',
974      sheetRow: 0,
975      unlock: { type: 'default' },
976    },
977    {
978      id: 'jelly',
979      category: 'orb',
980      name: '果冻弹珠',
981      description: '经典最高分 300 解锁',
982      sheetRow: 1,
983      unlock: { type: 'classicScore', score: 300 },
984    },
985    {
986      id: 'gem',
987      category: 'orb',
988      name: '宝石璀璨',
989      description: '通关到第 8 关解锁',
990      sheetRow: 2,
991      unlock: { type: 'levelReached', level: 8 },
992    },
993    {
994      id: 'candy',
995      category: 'orb',
996      name: '糖果缤纷',
997      description: '看广告解锁',
998      sheetRow: 3,
999      unlock: { type: 'ad', adUnitId: SKIN_REWARDED_AD_UNIT_ID },
1000  },
```

```
1001 {
1002   id: 'ocean',
1003   category: 'orb',
1004   name: '海洋之声',
1005   description: '通关到第 15 关解锁',
1006   sheetRow: 4,
1007   unlock: { type: 'levelReached', level: 15 },
1008 },
1009 {
1010   id: 'galaxy',
1011   category: 'orb',
1012   name: '星河流光',
1013   description: '通关到第 20 关解锁',
1014   sheetRow: 5,
1015   unlock: { type: 'levelReached', level: 20 },
1016 },
1017 {
1018   id: 'frost',
1019   category: 'orb',
1020   name: '冰晶雪花',
1021   description: '经典最高分 1000 解锁',
1022   sheetRow: 6,
1023   unlock: { type: 'classicScore', score: 1000 },
1024 },
1025 {
1026   id: 'flower',
1027   category: 'orb',
1028   name: '花漾琉璃',
1029   description: '看广告解锁',
1030   sheetRow: 7,
1031   unlock: { type: 'ad', adUnitId: SKIN_REWARDED_AD_UNIT_ID },
1032 },
1033 {
1034   id: 'lava',
1035   category: 'orb',
1036   name: '熔岩能量',
1037   description: '后续活动解锁',
1038   sheetRow: 8,
1039   unlock: { type: 'future', label: '后续活动解锁' },
1040 },
1041 ];
1042 export const BACKGROUND_SKINS: readonly BackgroundSkinDef[] = [
1043   {
1044     id: 'default-bg',
1045     category: 'background',
1046     name: '默认背景',
1047     description: '经典/关卡默认背景',
1048     imagePath: DEFAULT_LEVEL_BACKGROUND,
1049     previewPath: DEFAULT_LEVEL_BACKGROUND,
1050     unlock: { type: 'default' },
```

```
1051   },
1052   {
1053     id: 'arcade-wave',
1054     category: 'background',
1055     name: '碧蓝波纹',
1056     description: '到达第 4 关解锁',
1057     imagePath: 'subpkg_assets/images/bg_skin_arcade_wave_1k.png',
1058     previewPath: 'subpkg_assets/images/bg_skin_arcade_wave_1k.png',
1059     unlock: { type: 'levelReached', level: 4 },
1060   },
1061   {
1062     id: 'starlight-stream',
1063     category: 'background',
1064     name: '星辉流线',
1065     description: '到达第 8 关解锁',
1066     imagePath: 'subpkg_assets/images/bg_skin_starlight_stream_1k.png',
1067     previewPath: 'subpkg_assets/images/bg_skin_starlight_stream_1k.png',
1068     unlock: { type: 'levelReached', level: 8 },
1069   },
1070   {
1071     id: 'crystal-tide',
1072     category: 'background',
1073     name: '水晶潮汐',
1074     description: '经典 800 分解锁',
1075     imagePath: 'subpkg_assets/images/bg_skin_crystal_tide_1k.png',
1076     previewPath: 'subpkg_assets/images/bg_skin_crystal_tide_1k.png',
1077     unlock: { type: 'classicScore', score: 800 },
1078   },
1079   {
1080     id: 'neon-bubble',
1081     category: 'background',
1082     name: '霓虹泡泡',
1083     description: '到达第 12 关解锁',
1084     imagePath: 'subpkg_assets/images/bg_skin_neon_bubble_1k.png',
1085     previewPath: 'subpkg_assets/images/bg_skin_neon_bubble_1k.png',
1086     unlock: { type: 'levelReached', level: 12 },
1087   },
1088   {
1089     id: 'emerald-flow',
1090     category: 'background',
1091     name: '翠绿光流',
1092     description: '看广告解锁',
1093     imagePath: 'subpkg_assets/images/bg_skin_emerald_flow_1k.png',
1094     previewPath: 'subpkg_assets/images/bg_skin_emerald_flow_1k.png',
1095     unlock: { type: 'ad', adUnitId: SKIN_REWARDED_AD_UNIT_ID },
1096   },
1097   {
1098     id: 'sunset-splash',
1099     category: 'background',
1100     name: '暖橙浪花',
```

```

1101   description: '经典 1500 分解锁',
1102   imagePath: 'subpkg_assets/images/bg_skin_sunset_splash_1k.png',
1103   previewPath: 'subpkg_assets/images/bg_skin_sunset_splash_1k.png',
1104   unlock: { type: 'classicScore', score: 1500 },
1105 },
1106 {
1107   id: 'prism-orbit',
1108   category: 'background',
1109   name: '棱镜轨道',
1110   description: '看广告解锁',
1111   imagePath: 'subpkg_assets/images/bg_skin_prism_orbit_1k.png',
1112   previewPath: 'subpkg_assets/images/bg_skin_prism_orbit_1k.png',
1113   unlock: { type: 'ad', adUnitId: SKIN_REWARDED_AD_UNIT_ID },
1114 },
1115 {
1116   id: 'aurora-surf',
1117   category: 'background',
1118   name: '极光浪影',
1119   description: '后续活动解锁',
1120   imagePath: 'subpkg_assets/images/bg_skin_aurora_surf_1k.png',
1121   previewPath: 'subpkg_assets/images/bg_skin_aurora_surf_1k.png',
1122   unlock: { type: 'future', label: '后续活动解锁' },
1123 },
1124 {
1125   id: 'cloud-candy-dream',
1126   category: 'background',
1127   name: '棉花糖云空',
1128   description: '看广告解锁',
1129   imagePath: 'subpkg_assets/images/bg_skin_cloud_candy_2d_1k_test.png',
1130   previewPath: 'subpkg_assets/images/bg_skin_cloud_candy_2d_1k_test.png',
1131   unlock: { type: 'ad', adUnitId: SKIN_REWARDED_AD_UNIT_ID },
1132 },
1133 {
1134   id: 'warm-toy-glow',
1135   category: 'background',
1136   name: '暖光玩具桌',
1137   description: '看广告解锁',
1138   imagePath: 'subpkg_assets/images/bg_skin_warm_toy_1k_test.png',
1139   previewPath: 'subpkg_assets/images/bg_skin_warm_toy_1k_test.png',
1140   unlock: { type: 'ad', adUnitId: SKIN_REWARDED_AD_UNIT_ID },
1141 },
1142 ];
1143 export const ALL_SKINS: readonly SkinDef[] = [
1144   ...ORB_SKINS,
1145   ...BACKGROUND_SKINS,
1146 ];
1147 export function getSkinById(id: string): SkinDef | undefined {
1148   return ALL_SKINS.find((skin) => skin.id === id);
1149 }
1150 import { SEEN_SPECIAL_INTROS_KEY } from '@config/CloudConfig';

```

```
1151 import { PersistService } from '@core/PersistService';
1152 import { createNormalPiece, type Piece } from '@config/PieceConfig';
1153 export interface SpecialPieceIntroDef {
1154   id: string;
1155   levelId: number;
1156   title: string;
1157   iconPath?: string;
1158   iconPiece?: Piece;
1159   description: string;
1160   tip: string;
1161 }
1162 export const SPECIAL_PIECE_INTROS: readonly SpecialPieceIntroDef[] = [
1163   {
1164     id: 'color-green',
1165     levelId: 2,
1166     title: '绿色彩珠',
1167     iconPiece: createNormalPiece(3),
1168     description: '绿色彩珠加入棋盘。\\n同色五连可消除。',
1169     tip: '颜色变多，先找好连的颜色。',
1170   },
1171   {
1172     id: 'color-purple',
1173     levelId: 3,
1174     title: '紫色彩珠',
1175     iconPiece: createNormalPiece(4),
1176     description: '紫色彩珠加入棋盘。\\n可与其他紫色五连。',
1177     tip: '优先清集中色，别让盘太满。',
1178   },
1179   {
1180     id: 'color-orange',
1181     levelId: 4,
1182     title: '橙色彩珠',
1183     iconPiece: createNormalPiece(5),
1184     description: '橙色彩珠加入棋盘。\\n摆好位置凑五连。',
1185     tip: '多想想下一步路线。',
1186   },
1187   {
1188     id: 'color-cyan',
1189     levelId: 5,
1190     title: '青色彩珠',
1191     iconPiece: createNormalPiece(6),
1192     description: '青色彩珠加入棋盘。\\n七种颜色齐登场。',
1193     tip: '留出通路，别堵死棋盘。',
1194   },
1195   {
1196     id: 'wild',
1197     levelId: 6,
1198     title: '万能彩珠',
1199     iconPiece: { kind: 'wild' },
1200     description: '可代替任意颜色\\n参与同色五连。',
```



```
1201   tip: '放在关键格更好凑五连。',
1202 },
1203 {
1204   id: 'bomb',
1205   levelId: 7,
1206   title: '炸弹彩珠',
1207   iconPiece: { kind: 'bomb', color: 0 },
1208   description: '参与消除时\n爆破周围一圈。',
1209   tip: '连线引爆炸弹开局面。',
1210 },
1211 {
1212   id: 'frozen',
1213   levelId: 8,
1214   title: '冰冻彩珠',
1215   iconPiece: { kind: 'frozen', color: 2, layers: 1 },
1216   description: '不能移动, \n但可参与同色连线。',
1217   tip: '消除一次破冰变普通珠。',
1218 },
1219 {
1220   id: 'chain',
1221   levelId: 9,
1222   title: '锁链彩珠',
1223   iconPiece: { kind: 'chain', color: 5, layers: 1 },
1224   description: '被锁不能动, \n连线可破锁。',
1225   tip: '破锁后保留, 再连一次才消。',
1226 },
1227 {
1228   id: 'block',
1229   levelId: 10,
1230   title: '石块障碍',
1231   iconPiece: { kind: 'block', layers: 1 },
1232   description: '占格不动, \n不能参与连线。',
1233   tip: '炸弹可炸掉石块。',
1234 },
1235 ] as const;
1236 export function getSpecialPieceIntros(levelId: number): SpecialPieceIntroDef[] {
1237   return SPECIAL_PIECE_INTROS.filter(intro => intro.levelId === levelId);
1238 }
1239 export function hasSeenSpecialPieceIntro(id: string): boolean {
1240   const seen = _loadSeenIntros();
1241   return seen.includes(id);
1242 }
1243 export function markSpecialPieceIntroSeen(id: string): void {
1244   const seen = new Set(_loadSeenIntros());
1245   seen.add(id);
1246   PersistService.writeJSON(SEEN_SPECIAL_INTROS_KEY, [...seen]);
1247 }
1248 function _loadSeenIntros(): string[] {
1249   const parsed = PersistService.readJSON<unknown>(SEEN_SPECIAL_INTROS_KEY);
1250   return Array.isArray(parsed) ? parsed.filter(item => typeof item === 'string') : [];
```

```
1251 }
1252 declare const wx: any;
1253 declare const tt: any;
1254 const _api = typeof wx !== 'undefined' ? wx : typeof tt !== 'undefined' ? tt : null;
1255 const THROTTLE_MS = 50;
1256 const MUSIC_MUTED_KEY = 'caizhu_audio_music_muted';
1257 const SFX_MUTED_KEY = 'caizhu_audio_sfx_muted';
1258 interface SoundEntry { src: string; volume: number; }
1259 class AudioManagerClass {
1260   private _sounds: Map<string, SoundEntry> = new Map();
1261   private _bgm: any = null;
1262   private _musicMuted = this._readMuted(MUSIC_MUTED_KEY);
1263   private _sfxMuted = this._readMuted(SFX_MUTED_KEY);
1264   private _bgmPending: { src: string; volume: number } | null = null;
1265   private _bgmSrc = "";
1266   private _lastPlayTime: Map<string, number> = new Map();
1267   register(name: string, src: string, volume = 1): void {
1268     this._sounds.set(name, { src, volume });
1269   }
1270   play(name: string): void {
1271     if (this._sfxMuted || !_api) return;
1272     const entry = this._sounds.get(name);
1273     if (!entry) return;
1274     const now = Date.now();
1275     const last = this._lastPlayTime.get(name) || 0;
1276     if (now - last < THROTTLE_MS) return;
1277     this._lastPlayTime.set(name, now);
1278     try {
1279       const audio = _api.createInnerAudioContext();
1280       if (!audio) return;
1281       let done = false;
1282       const cleanup = () => { if (done) return; done = true; try { audio.destroy(); } catch {} };
1283       audio.volume = entry.volume;
1284       audio.onError(() => cleanup());
1285       audio.onEnded(() => cleanup());
1286       audio.src = entry.src;
1287       try { audio.play(); } catch { cleanup(); }
1288     } catch {}
1289   }
1290   playBGM(src: string, volume = 0.5): void {
1291     if (!_api) return;
1292     if (this._bgm && this._bgmSrc === src) {
1293       try { this._bgm.volume = volume; } catch {}
1294       if (!this._musicMuted) {
1295         try { this._bgm.play(); } catch {}
1296       }
1297       return;
1298     }
1299     if (!this._bgm && this._bgmPending?.src === src) {
1300       this._bgmPending.volume = volume;
```

```
1301     return;
1302 }
1303 this.stopBGM();
1304 this._bgmPending = { src, volume };
1305 try {
1306     this._bgm = _api.createInnerAudioContext();
1307     if (!this._bgm) return;
1308     this._bgmSrc = src;
1309     this._bgm.loop = true;
1310     this._bgm.volume = volume;
1311     this._bgm.onError() => {
1312         try { this._bgm?.destroy?(); } catch {}
1313         this._bgm = null;
1314         this._bgmSrc = "";
1315     });
1316     this._bgm.onPlay() => { this._bgmPending = null; };
1317     this._bgm.src = src;
1318     if (!this._musicMuted) {
1319         try { this._bgm.play(); } catch {}
1320     }
1321     } catch { this._bgm = null; }
1322 }
1323 resumeOnInteraction(): void {
1324     if (this._musicMuted) return;
1325     if (this._bgmPending && !this._bgm) {
1326         this.playBGM(this._bgmPending.src, this._bgmPending.volume);
1327     } else if (this._bgm && this._bgmPending) {
1328         try { this._bgm.play(); } catch {}
1329         this._bgmPending = null;
1330     }
1331 }
1332 stopBGM(): void {
1333     if (this._bgm) {
1334         try { this._bgm.stop(); this._bgm.destroy(); } catch {}
1335         this._bgm = null;
1336     }
1337     this._bgmSrc = "";
1338 }
1339 get muted(): boolean { return this._musicMuted && this._sfxMuted; }
1340 set muted(val: boolean) {
1341     this.musicMuted = val;
1342     this.sfxMuted = val;
1343 }
1344 get musicMuted(): boolean { return this._musicMuted; }
1345 set musicMuted(val: boolean) {
1346     this._musicMuted = val;
1347     this._writeMuted(MUSIC_MUTED_KEY, val);
1348     if (this._bgm) { try { val ? this._bgm.pause() : this._bgm.play(); } catch {} }
1349 }
1350 get sfxMuted(): boolean { return this._sfxMuted; }
```

```
1351 set sfxMuted(val: boolean) {
1352   this._sfxMuted = val;
1353   this._writeMuted(SFX_MUTED_KEY, val);
1354 }
1355 private _readMuted(key: string): boolean {
1356   try {
1357     const api = _api || (typeof localStorage !== 'undefined' ? localStorage : null);
1358     const value = api?.getStorageSync ? api.getStorageSync(key) : api?.getItem?.(key);
1359     return value === '1' || value === 'true';
1360   } catch {
1361     return false;
1362   }
1363 }
1364 private _writeMuted(key: string, muted: boolean): void {
1365   try {
1366     const api = _api || (typeof localStorage !== 'undefined' ? localStorage : null);
1367     const value = muted ? '1' : '0';
1368     if (api?.setStorageSync) api.setStorageSync(key, value);
1369     else api?.setItem?.(key, value);
1370   } catch {}
1371 }
1372 }
1373 export const AudioManager = new AudioManagerClass();
1374 import {
1375   BACKEND_ANON_ID_KEY,
1376   BACKEND_BASE_URL,
1377   BACKEND_LB_CLASSIC_SUBMIT_PATH,
1378   BACKEND_LB_CLASSIC_WORLD_PATH,
1379   BACKEND_LB_LEVEL_SUBMIT_PATH,
1380   BACKEND_LB_LEVEL_WORLD_PATH,
1381   BACKEND_LOGIN_PATH,
1382   BACKEND_PROFILE_GET_PATH,
1383   BACKEND_PROFILE_UPDATE_PATH,
1384   BACKEND_PULL_PATH,
1385   BACKEND_PUSH_PATH,
1386   BACKEND_REQUEST_TIMEOUT_MS,
1387   BACKEND_TOKEN_KEY,
1388 } from '@config/CloudConfig';
1389 import { Platform } from '@core/PlatformService';
1390 export interface BackendPullResult {
1391   userId: string;
1392   platform: string;
1393   exists: boolean;
1394   schemaVersion: number;
1395   updatedAt: number;
1396   payload: Record<string, string>;
1397   payloadKeys: string[];
1398   clientFingerprint?: string;
1399 }
1400 export interface BackendPushPayload {
```

```
1401  schemaVersion: number;
1402  updatedAt: number;
1403  baseRemoteUpdatedAt: number;
1404  clientFingerprint: string;
1405  payload: Record<string, string>;
1406  }
1407  export interface BackendPushResult {
1408    userId: string;
1409    updatedAt: number;
1410    savedAt: number;
1411    mode: 'insert' | 'update';
1412    sizeBytes: number;
1413  }
1414  export interface LeaderboardClassicEntry {
1415    rank: number;
1416    isMe: boolean;
1417    userId: string;
1418    nickname: string;
1419    avatarUrl: string;
1420    bestScore: number;
1421    updatedAt: number;
1422  }
1423  export interface LeaderboardLevelEntry {
1424    rank: number;
1425    isMe: boolean;
1426    userId: string;
1427    nickname: string;
1428    avatarUrl: string;
1429    totalStars: number;
1430    totalScore: number;
1431    maxUnlocked: number;
1432    updatedAt: number;
1433  }
1434  export interface LeaderboardFetchMeta {
1435    ok: boolean;
1436    code?: string;
1437    message?: string;
1438  }
1439  export interface LeaderboardWorldResult<T> {
1440    items: T[];
1441    me: T | null;
1442    total: number;
1443    /** 全服榜接口拉取状态; 失败时 items 为空, 不应把本地成绩混入列表冒充全服榜 */
1444    fetch?: LeaderboardFetchMeta;
1445  }
1446  export interface LeaderboardSubmitProfile {
1447    nickname?: string;
1448    avatarUrl?: string;
1449  }
1450  export interface ClassicSubmitResult {
```

```
1451   userId: string;
1452   bestScore: number;
1453   savedAt: number;
1454   improved: boolean;
1455 }
1456 export interface LevelSubmitResult {
1457   userId: string;
1458   totalStars: number;
1459   totalScore: number;
1460   maxUnlocked: number;
1461   savedAt: number;
1462   improved: boolean;
1463 }
1464 export interface ProfileUpdateResult {
1465   userId: string;
1466   nickname: string;
1467   avatarUrl: string;
1468   savedAt: number;
1469 }
1470 interface StoredToken {
1471   token: string;
1472   userId: string;
1473   platform: string;
1474   expiresAt: number;
1475 }
1476 export class BackendError extends Error {
1477   readonly status: number;
1478   readonly code: string;
1479   readonly data?: any;
1480   constructor(status: number, code: string, message: string, data?: any) {
1481     super(message);
1482     this.name = 'BackendError';
1483     this.status = status;
1484     this.code = code;
1485     this.data = data;
1486   }
1487 }
1488 class BackendServiceClass {
1489   private stored: StoredToken | null = null;
1490   private loginInflight: Promise<StoredToken> | null = null;
1491   get available(): boolean {
1492     return Platform.canUseBackend;
1493   }
1494   get userId(): string {
1495     return this.stored?.userId || '';
1496   }
1497   async ensureToken(): Promise<StoredToken> {
1498     if (this.stored && this.stored.expiresAt - Date.now() > 60_000) {
1499       return this.stored;
1500     }
```

```

14486     updatedAt,
14487     savedAt: now,
14488     mode: 'insert',
14489     sizeBytes: size,
14490     _id: addRes && (addRes.id || addRes._id),
14491   };
14492 }
14493 module.exports = {
14494   handlePull,
14495   handlePush,
14496 };
14497 // 好友榜: canvas 尺寸只读, 按主域传入的 listWidth / rowHeight 换算像素布局
14498 var utils = require('./utils.js');
14499 var DESIGN_W = 750;
14500 var ROW_PANEL = 'images/rank_row_panel.png';
14501 var ROW_PANEL_BORDER = { left: 64, top: 20, right: 64, bottom: 20 };
14502 var LIST_TOP = 8;
14503 var PADDING_X = 30;
14504 var FONT_FAMILY = 'PingFang SC, Hiragino Sans GB, Microsoft YaHei, Arial, sans-serif';
14505 var IMAGE_CACHE = {};
14506 var ASSETS = { rowPanel: null, rowReady: false, rowLoading: false };
14507 function setRedrawTrigger(redrawFn) {
14508   IMAGE_CACHE.__onLoad = redrawFn;
14509 }
14510 function clearCache() {
14511   for (var k in IMAGE_CACHE) {
14512     if (k !== '__onLoad') delete IMAGE_CACHE[k];
14513   }
14514 }
14515 function prepCtx(ctx) {
14516   ctx.setTransform(1, 0, 0, 1, 0, 0);
14517   ctx.imageSmoothingEnabled = true;
14518   if (typeof ctx.imageSmoothingQuality === 'string') {
14519     ctx.imageSmoothingQuality = 'high';
14520   }
14521 }
14522 function clearCanvas(ctx, w, h) {
14523   ctx.clearRect(0, 0, w, h);
14524 }
14525 /**
14526  * 与全服榜行高对齐: 主域显示时 scale=listW/cw, 故 canvas 上 rowH = targetRowH * cw / listW。
14527  *
14528  * 列布局采用「相对前一列依次排版」策略, 避免按 k 各自缩放后互相穿插。
14529  * 所有 X 都是「相对行起点 x」的偏移 (除 valueX 是 canvasW 上的绝对 x, 与原代码保持兼容)。
14530  * rankX      : 名次列居中 x (相对 x)
14531  * avatarCx   : 头像圆心 x (相对 x)
14532  * nameX      : 昵称左对齐 x (相对 x)
14533  * valueX     : 分数右对齐 x (canvas 绝对 x)
14534  */
14535 function getLayout(canvasW, listW, targetRowH) {

```

```
14536 var k = canvasW > 0 ? canvasW / DESIGN_W : 1;
14537 var rowH = Math.max(40, Math.round(targetRowH * canvasW / listW));
14538 var rowGap = Math.max(4, Math.round(8 * k));
14539 var pad = Math.max(12, Math.round(PADDING_X * k));
14540 var avatarR = Math.max(16, Math.round(24 * k));
14541 var rankColW = Math.max(36, Math.round(48 * k));
14542 var rankX = Math.round(rankColW / 2);
14543 var avatarCx = rankColW + avatarR + Math.max(6, Math.round(8 * k));
14544 var nameX = avatarCx + avatarR + Math.max(12, Math.round(16 * k));
14545 var valueX = canvasW - pad - Math.max(6, Math.round(8 * k));
14546 return {
14547   pad: pad,
14548   rowH: rowH,
14549   rowGap: rowGap,
14550   rowW: canvasW - pad * 2,
14551   avatarR: avatarR,
14552   avatarCx: avatarCx,
14553   rankX: rankX,
14554   nameX: nameX,
14555   valueX: valueX,
14556   fontRankMedal: Math.max(18, Math.round(20 * k)) + 'px' + FONT_FAMILY,
14557   fontRankNum: 'bold' + Math.max(14, Math.round(16 * k)) + 'px' + FONT_FAMILY,
14558   fontName: 'bold' + Math.max(15, Math.round(17 * k)) + 'px' + FONT_FAMILY,
14559   fontValue: 'bold' + Math.max(16, Math.round(18 * k)) + 'px' + FONT_FAMILY,
14560   fontUnit: Math.max(10, Math.round(11 * k)) + 'px' + FONT_FAMILY,
14561 };
14562 }
14563 function computeContentHeight(entryCount, layout) {
14564   var n = Math.max(0, entryCount || 0);
14565   if (n === 0) return 120;
14566   return LIST_TOP + n * (layout.rowH + layout.rowGap) + 16;
14567 }
14568 function loadRowPanel(done) {
14569   if (ASSETS.rowReady && ASSETS.rowPanel) {
14570     done(true);
14571     return;
14572   }
14573   if (ASSETS.rowLoading) {
14574     done(false);
14575     return;
14576   }
14577   if (!wx || !wx.createImage) {
14578     done(false);
14579     return;
14580   }
14581   ASSETS.rowLoading = true;
14582   var img = wx.createImage();
14583   img.onload = function () {
14584     ASSETS.rowPanel = img;
14585     ASSETS.rowReady = true;
```



```
14586  ASSETS.rowLoading = false;
14587  done(true);
14588  };
14589  img.onerror = function () {
14590    ASSETS.rowReady = false;
14591    ASSETS.rowLoading = false;
14592    done(false);
14593  };
14594  img.src = ROW_PANEL;
14595  }
14596  function drawNineSlice(ctx, img, x, y, w, h, border) {
14597    var iw = img.width;
14598    var ih = img.height;
14599    var l = border.left;
14600    var t = border.top;
14601    var r = border.right;
14602    var b = border.bottom;
14603    var cw = iw - l - r;
14604    var ch = ih - t - b;
14605    var dw = w - l - r;
14606    var dh = h - t - b;
14607    ctx.drawImage(img, 0, 0, l, t, x, y, l, t);
14608    ctx.drawImage(img, l, 0, cw, t, x + l, y, dw, t);
14609    ctx.drawImage(img, iw - r, 0, r, t, x + w - r, y, r, t);
14610    ctx.drawImage(img, 0, t, l, ch, x, y + t, l, dh);
14611    ctx.drawImage(img, l, t, cw, ch, x + l, y + t, dw, dh);
14612    ctx.drawImage(img, iw - r, t, r, ch, x + w - r, y + t, r, dh);
14613    ctx.drawImage(img, 0, ih - b, l, b, x, y + h - b, l, b);
14614    ctx.drawImage(img, l, ih - b, cw, b, x + l, y + h - b, dw, b);
14615    ctx.drawImage(img, iw - r, ih - b, r, b, x + w - r, y + h - b, r, b);
14616  }
14617  function drawRowPanelFallback(ctx, x, y, w, h, isMe) {
14618    roundRect(ctx, x, y, w, h, 12);
14619    var grad = ctx.createLinearGradient(x, y, x, y + h);
14620    if (isMe) {
14621      grad.addColorStop(0, '#FFF9E6');
14622      grad.addColorStop(1, '#FFF3C4');
14623    } else {
14624      grad.addColorStop(0, '#FFFFFF');
14625      grad.addColorStop(1, '#E8F4FF');
14626    }
14627    ctx.fillStyle = grad;
14628    ctx.fill();
14629    ctx.strokeStyle = isMe ? 'rgba(250, 204, 21, 0.9)' : 'rgba(96, 165, 250, 0.75)';
14630    ctx.lineWidth = 2;
14631    ctx.stroke();
14632  }
14633  function roundRect(ctx, x, y, w, h, r) {
14634    ctx.beginPath();
14635    ctx.moveTo(x + r, y);
```

```
14636 ctx.lineTo(x + w - r, y);
14637 ctx.quadraticCurveTo(x + w, y, x + w, y + r);
14638 ctx.lineTo(x + w, y + h - r);
14639 ctx.quadraticCurveTo(x + w, y + h, x + w - r, y + h);
14640 ctx.lineTo(x + r, y + h);
14641 ctx.quadraticCurveTo(x, y + h, x, y + h - r);
14642 ctx.lineTo(x, y + r);
14643 ctx.quadraticCurveTo(x, y, x + r, y);
14644 ctx.closePath();
14645 }
14646 function strokeFillText(ctx, text, x, y, fill, stroke, lineW, font) {
14647     ctx.font = font;
14648     ctx.textAlign = 'center';
14649     ctx.textBaseline = 'middle';
14650     ctx.lineWidth = lineW;
14651     ctx.strokeStyle = stroke;
14652     ctx.fillStyle = fill;
14653     ctx.strokeText(text, x, y);
14654     ctx.fillText(text, x, y);
14655 }
14656 function drawCompactRow(ctx, x, y, layout, entry, rank, isMe, metricLabel) {
14657     var w = layout.rowW;
14658     var h = layout.rowH;
14659     if (ASSETS.rowReady && ASSETS.rowPanel) {
14660         drawNineSlice(ctx, ASSETS.rowPanel, x, y, w, h, ROW_PANEL_BORDER);
14661     } else {
14662         drawRowPanelFallback(ctx, x, y, w, h, isMe);
14663     }
14664     var cy = y + h / 2;
14665     ctx.save();
14666     if (rank <= 3) {
14667         ctx.font = layout.fontRankMedal;
14668         ctx.textAlign = 'center';
14669         ctx.textBaseline = 'middle';
14670         ctx.fillText(rank === 1 ? '?' : rank === 2 ? '?' : '?', x + layout.rankX, cy);
14671     } else {
14672         strokeFillText(
14673             ctx,
14674             String(rank),
14675             x + layout.rankX,
14676             cy,
14677             '#0B4A8B',
14678             'FFFFFF',
14679             2,
14680             layout.fontRankNum,
14681         );
14682     }
14683     ctx.restore();
14684     var avatarX = x + layout.avatarCx;
14685     utils.drawCircleAvatar(ctx, entry.avatarUrl, avatarX, cy, layout.avatarR, IMAGE_CACHE, entry.ni...
```

```
14686   ctx.save();
14687   ctx.fillStyle = '#102F64';
14688   ctx.font = layout.fontName;
14689   ctx.textAlign = 'left';
14690   ctx.textBaseline = 'middle';
14691   var name = utils.truncateNickname(entry.nickname || '微信好友', 8);
14692   ctx.fillText(name, x + layout.nameX, cy);
14693   ctx.restore();
14694   if (isMe) {
14695     ctx.save();
14696     var tagX = x + layout.nameX + ctx.measureText(name).width + 4;
14697     ctx.fillStyle = '#FACC15';
14698     roundRect(ctx, tagX, cy - 8, 20, 15, 4);
14699     ctx.fill();
14700     ctx.fillStyle = '#1F2937';
14701     ctx.font = 'bold 10px ' + FONT_FAMILY;
14702     ctx.textAlign = 'center';
14703     ctx.textBaseline = 'middle';
14704     ctx.fillText('我', tagX + 10, cy);
14705     ctx.restore();
14706   }
14707   var valueText = utils.formatNumber(entry.value) + (metricLabel === '星' ? '星' : '分');
14708   ctx.save();
14709   ctx.fillStyle = '#0B4A8B';
14710   ctx.font = layout.fontValue;
14711   ctx.textAlign = 'right';
14712   ctx.textBaseline = 'middle';
14713   ctx.lineWidth = 2;
14714   ctx.strokeStyle = '#FFFFFF';
14715   ctx.strokeText(valueText, layout.valueX, cy);
14716   ctx.fillText(valueText, layout.valueX, cy);
14717   ctx.restore();
14718 }
14719 function drawTopMessage(ctx, w, h, message, layout) {
14720   prepCtx(ctx);
14721   clearCanvas(ctx, w, h);
14722   ctx.fillStyle = 'rgba(148,163,184,0.9)';
14723   ctx.font = layout ? layout.fontName : ('16px ' + FONT_FAMILY);
14724   ctx.textAlign = 'center';
14725   ctx.textBaseline = 'top';
14726   var lines = String(message || '').split('\n');
14727   var lineH = 24;
14728   var startY = LIST_TOP + 8;
14729   for (var i = 0; i < lines.length; i++) {
14730     ctx.fillText(lines[i], w / 2, startY + i * lineH);
14731   }
14732 }
14733 function drawFriendList(ctx, w, h, entries, metricLabel, listW, targetRowH, outCanvas) {
14734   var layout = getLayout(w, listW, targetRowH);
14735   if (!entries || entries.length === 0) {
```

```
14736 drawTopMessage(ctx, w, h, '还没有好友数据\n邀请好友一起玩吧', layout);
14737 if (outCanvas) outCanvas.contentHeight = 120;
14738 return;
14739 }
14740 function paint() {
14741   prepCtx(ctx);
14742   clearCanvas(ctx, w, h);
14743   for (var i = 0; i < entries.length; i++) {
14744     var rowY = LIST_TOP + i * (layout.rowH + layout.rowGap);
14745     drawCompactRow(ctx, layout.pad, rowY, layout, entries[i], i + 1, entries[i].isMe, metricLab...
14746   }
14747   if (outCanvas) {
14748     outCanvas.contentHeight = computeContentHeight(entries.length, layout);
14749   }
14750 }
14751 loadRowPanel(function () {
14752   paint();
14753 });
14754 }
14755 module.exports = {
14756   drawFriendList: drawFriendList,
14757   drawEmptyState: drawTopMessage,
14758   setRedrawTrigger: setRedrawTrigger,
14759   clearCache: clearCache,
14760   computeContentHeight: computeContentHeight,
14761   getLayout: getLayout,
14762   LIST_TOP: LIST_TOP,
14763 };
14764 // 开放数据域：拉取好友云存储并绘制到 sharedCanvas (canvas 宽高只读)
14765 var drawer = require('./drawer.js');
14766 var WX = (typeof wx !== 'undefined') ? wx : null;
14767 var sharedCanvas = WX && WX.getSharedCanvas ? WX.getSharedCanvas() : null;
14768 var ctx = sharedCanvas ? sharedCanvas.getContext('2d') : null;
14769 // 真机诊断：开放数据域脚本被加载到的时刻、sharedCanvas size、ctx 是否拿到
14770 try {
14771   console.log(
14772     '[RankDiag-Sub] open_data loaded'
14773     + ' hasWX=' + !!WX
14774     + ' getFriendCloudStorage=' + (WX && typeof WX.getFriendCloudStorage === 'function')
14775     + ' sharedCanvas=' + !!sharedCanvas
14776     + ' size=' + (sharedCanvas ? (sharedCanvas.width | 0) + 'x' + (sharedCanvas.height | 0) : '...
14777     + ' ctx=' + !!ctx
14778   );
14779 } catch (_) {}
14780 var state = {
14781   tab: 'classic',
14782   metric: 'classic_best',
14783   metricLabel: '分',
14784   listWidth: 690,
14785   rowHeight: 76,
```

```
14786   lastEntries: null,
14787   fetchInflight: false,
14788 };
14789 function readCanvasSize() {
14790   if (!sharedCanvas) return { width: 0, height: 0 };
14791   return { width: sharedCanvas.width, height: sharedCanvas.height };
14792 }
14793 function publishContentHeight(h) {
14794   if (sharedCanvas) {
14795     sharedCanvas.__friendContentHeight = h;
14796   }
14797 }
14798 function redrawLast() {
14799   if (!state.lastEntries || !ctx || !sharedCanvas) return;
14800   var size = readCanvasSize();
14801   var meta = {};
14802   drawer.drawFriendList(
14803     ctx,
14804     size.width,
14805     size.height,
14806     state.lastEntries,
14807     state.metricLabel,
14808     state.listWidth,
14809     state.rowHeight,
14810     meta,
14811   );
14812   publishContentHeight(meta.contentHeight || 120);
14813 }
14814 drawer.setRedrawTrigger(redrawLast);
14815 function metricOf(tab) {
14816   if (tab === 'level') {
14817     return { metric: 'level_stars', label: '星' };
14818   }
14819   return { metric: 'classic_best', label: '分' };
14820 }
14821 function safeNumber(s) {
14822   var n = Number(s);
14823   return isFinite(n) ? n : 0;
14824 }
14825 function pickValue(KVDataList, key) {
14826   if (!KVDataList || !KVDataList.length) return 0;
14827   for (var i = 0; i < KVDataList.length; i++) {
14828     if (KVDataList[i] && KVDataList[i].key === key) {
14829       return safeNumber(KVDataList[i].value);
14830     }
14831   }
14832   return 0;
14833 }
14834 function buildEntries(items, metric, selfOpenId) {
14835   var list = [];
```

```
14836 for (var i = 0; i < items.length; i++) {
14837     var it = items[i] || {};
14838     var value = pickValue(it.KVDataList, metric);
14839     if (value <= 0) continue;
14840     list.push({
14841         openid: it.openid || "",
14842         nickname: it.nickname || "",
14843         avatarUrl: it.avatarUrl || "",
14844         value: value,
14845         isMe: !(selfOpenId && it.openid === selfOpenId),
14846     });
14847 }
14848 list.sort(function (a, b) { return b.value - a.value; });
14849 return list;
14850 }
14851 function fetchAndRender() {
14852     if (!WX || !ctx || !sharedCanvas) {
14853         try {
14854             console.log('[RankDiag-Sub] fetchAndRender abort hasWX=' + !!WX + ' hasCtx=' + !!ctx + ' ha...
14855         } catch (_) {}
14856         return;
14857     }
14858     if (state.fetchInflight) return;
14859     state.fetchInflight = true;
14860     var size = readCanvasSize();
14861     var meta = {};
14862     drawer.drawEmptyState(ctx, size.width, size.height, '加载好友榜中...');
14863     publishContentHeight(80);
14864     var pickedMetric = state.metric;
14865     var pickedLabel = state.metricLabel;
14866     var selfOpenId = "";
14867     function doFetch() {
14868         if (!WX.getFriendCloudStorage) {
14869             state.fetchInflight = false;
14870             drawer.drawEmptyState(ctx, size.width, size.height, '当前环境不支持好友榜');
14871             publishContentHeight(80);
14872             return;
14873         }
14874         try {
14875             console.log('[RankDiag-Sub] getFriendCloudStorage metric=' + pickedMetric);
14876         } catch (_) {}
14877         WX.getFriendCloudStorage({
14878             keyList: [pickedMetric],
14879             success: function (res) {
14880                 var entries = buildEntries((res && res.data) || [], pickedMetric, selfOpenId);
14881                 state.lastEntries = entries;
14882                 size = readCanvasSize();
14883                 try {
14884                     console.log(
14885                         '[RankDiag-Sub] getFriendCloudStorage success'
```

```
14886         + 'raw=' + ((res && res.data) ? res.data.length : 0)
14887         + 'entries=' + entries.length
14888         + 'canvas=' + size.width + 'x' + size.height
14889     );
14890 } catch (_) {}
14891 meta = {};
14892 drawer.drawFriendList(
14893     ctx,
14894     size.width,
14895     size.height,
14896     entries,
14897     pickedLabel,
14898     state.listWidth,
14899     state.rowHeight,
14900     meta,
14901 );
14902 publishContentHeight(meta.contentHeight || 120);
14903 },
14904 fail: function (err) {
14905     var msg = '好友榜读取失败';
14906     if (err && err.errMsg) {
14907         msg = '好友榜读取失败\n' + String(err.errMsg).slice(0, 40);
14908     }
14909     try {
14910         console.log('[RankDiag-Sub] getFriendCloudStorage fail ' + (err && err.errMsg));
14911     } catch (_) {}
14912     size = readCanvasSize();
14913     drawer.drawEmptyState(ctx, size.width, size.height, msg);
14914     publishContentHeight(80);
14915 },
14916 complete: function () {
14917     state.fetchInflight = false;
14918 },
14919 });
14920 }
14921 if (WX.getUserInfo) {
14922     try {
14923         WX.getUserInfo({
14924             openIdList: ['selfOpenId'],
14925             success: function (info) {
14926                 var d = info && info.data && info.data[0];
14927                 if (d && d.openId) selfOpenId = d.openId;
14928                 doFetch();
14929             },
14930             fail: function () { doFetch(); },
14931         });
14932         return;
14933     } catch (_) {}
14934 }
14935 doFetch();
```

```
14936 }
14937 if (WX && WX.onMessage) {
14938   WX.onMessage(function (msg) {
14939     if (!msg || typeof msg !== 'object') return;
14940     var type = String(msg.type || '');
14941     try {
14942       console.log('[RankDiag-Sub] onMessage type=' + type + ' tab=' + msg.tab + ' listWidth=' + m...
14943     } catch (_) {}
14944     if (type === 'render') {
14945       var tab = String(msg.tab || 'classic');
14946       var pair = metricOf(tab);
14947       state.tab = tab;
14948       state.metric = pair.metric;
14949       state.metricLabel = pair.label;
14950       if (msg.listWidth) state.listWidth = Number(msg.listWidth);
14951       if (msg.rowHeight) state.rowHeight = Number(msg.rowHeight);
14952       fetchAndRender();
14953     } else if (type === 'clearCache') {
14954       drawer.clearCache();
14955     }
14956   });
14957 }
14958 if (ctx && sharedCanvas) {
14959   var initSize = readCanvasSize();
14960   drawer.drawEmptyState(ctx, initSize.width, initSize.height, '好友榜准备中');
14961   publishContentHeight(80);
14962 }
14963 // 开放数据域共用工具：截断昵称、绘制圆形头像（带占位图）、格式化数字。
14964 function truncateNickname(name, max) {
14965   var s = String(name || '').trim();
14966   if (!s) return '';
14967   if (s.length <= max) return s;
14968   return s.slice(0, max) + '...';
14969 }
14970 function formatNumber(n) {
14971   var v = Number(n || 0);
14972   if (!isFinite(v)) return '0';
14973   return String(Math.floor(v));
14974 }
14975 // 在指定圆形区域内绘制头像；若头像不可用则绘制占位渐变。
14976 // imageCache: 用于缓存 Image 对象，避免重复创建
14977 function drawCircleAvatar(ctx, url, cx, cy, radius, imageCache, fallbackText) {
14978   ctx.save();
14979   ctx.beginPath();
14980   ctx.arc(cx, cy, radius, 0, Math.PI * 2, false);
14981   ctx.closePath();
14982   ctx.clip();
14983   if (url) {
14984     var cached = imageCache[url];
14985     if (!cached) {
```



```
14986     try {
14987         var img = wx.createImage();
14988         img.onload = function () {
14989             imageCache[url] = { ready: true, img: img };
14990             if (typeof imageCache.__onLoad === 'function') {
14991                 imageCache.__onLoad();
14992             }
14993         };
14994         img.onerror = function () {
14995             imageCache[url] = { ready: false, error: true };
14996         };
14997         img.src = url;
14998         imageCache[url] = { ready: false, img: img };
14999     } catch (_) {
15000         imageCache[url] = { ready: false, error: true };
15001     }
15002 } else if (cached.ready && cached.img) {
15003     try {
15004         ctx.drawImage(cached.img, cx - radius, cy - radius, radius * 2, radius * 2);
15005         ctx.restore();
15006         // 圆形描边
15007         ctx.beginPath();
15008         ctx.arc(cx, cy, radius, 0, Math.PI * 2, false);
15009         ctx.lineWidth = 2;
15010         ctx.strokeStyle = 'rgba(255,255,255,0.45)';
15011         ctx.stroke();
15012         return;
15013     } catch (_) {}
15014 }
15015 }
15016 // Placeholder: gradient circle + initial letter.
15017 var grad = ctx.createLinearGradient(cx - radius, cy - radius, cx + radius, cy + radius);
15018 grad.addColorStop(0, '#5b8cff');
15019 grad.addColorStop(1, '#a23bff');
15020 ctx.fillStyle = grad;
15021 ctx.beginPath();
15022 ctx.arc(cx, cy, radius, 0, Math.PI * 2, false);
15023 ctx.fill();
15024 var letter = String(fallbackText || 'G').charAt(0).toUpperCase();
15025 ctx.fillStyle = '#FFFFFF';
15026 ctx.font = 'bold ' + Math.floor(radius) + 'px sans-serif';
15027 ctx.textAlign = 'center';
15028 ctx.textBaseline = 'middle';
15029 ctx.fillText(letter, cx, cy + 1);
15030 ctx.restore();
15031 ctx.beginPath();
15032 ctx.arc(cx, cy, radius, 0, Math.PI * 2, false);
15033 ctx.lineWidth = 2;
15034 ctx.strokeStyle = 'rgba(255,255,255,0.45)';
15035 ctx.stroke();
```

```
15036 }
15037 module.exports = {
15038   truncateNickname: truncateNickname,
15039   formatNumber: formatNumber,
15040   drawCircleAvatar: drawCircleAvatar,
15041 };
15042 /**
15043  * Image 构造函数模拟
15044  */
15045 const platform = require('./platform');
15046 function Image() {
15047   return platform.createImage();
15048 }
15049 module.exports = Image;
15050 /**
15051  * 触摸事件适配
15052  * 将小游戏触摸事件转换为 PixiJS 所需的标准 DOM 事件格式
15053  */
15054 const platform = require('./platform');
15055 const { canvas } = require('./canvas');
15056 class TouchEvent {
15057   constructor(type, touches) {
15058     this.type = type;
15059     this.target = canvas;
15060     this.currentTarget = canvas;
15061     this.touches = touches || [];
15062     this.changedTouches = touches || [];
15063     this.targetTouches = touches || [];
15064     this.timeStamp = Date.now();
15065     this.bubbles = true;
15066     this.cancelable = true;
15067     this.defaultPrevented = false;
15068   }
15069   preventDefault() {
15070     this.defaultPrevented = true;
15071   }
15072   stopPropagation() {}
15073 }
15074 // 将小游戏触摸坐标转换为 canvas 坐标
15075 function convertTouches(rawTouches) {
15076   if (!rawTouches) return [];
15077   return Array.prototype.map.call(rawTouches, (touch) => ({
15078     identifier: touch.identifier,
15079     clientX: touch.clientX,
15080     clientY: touch.clientY,
15081     pageX: touch.clientX,
15082     pageY: touch.clientY,
15083     screenX: touch.clientX,
15084     screenY: touch.clientY,
15085     target: canvas,
```

```
15086   });
15087 }
15088 // 注册触摸事件监听桥接
15089 function registerTouchEvents() {
15090   const _listeners = {};
15091   // 给 canvas 挂上 addEventListener / removeEventListener
15092   canvas.addEventListener = function(type, handler, options) {
15093     if (!_listeners[type]) _listeners[type] = [];
15094     _listeners[type].push(handler);
15095   };
15096   canvas.removeEventListener = function(type, handler) {
15097     if (!_listeners[type]) return;
15098     const idx = _listeners[type].indexOf(handler);
15099     if (idx !== -1) _listeners[type].splice(idx, 1);
15100   };
15101   // 分发事件到 canvas 上的监听器
15102   function dispatch(type, rawEvent) {
15103     const touches = convertTouches(rawEvent.touches || rawEvent.changedTouches);
15104     const event = new TouchEvent(type, touches);
15105     event.changedTouches = convertTouches(rawEvent.changedTouches);
15106     const queue = _listeners[type];
15107     if (queue) {
15108       queue.forEach(handler => {
15109         try { handler(event); } catch (e) { console.error('[TouchEvent]', type, e); }
15110       });
15111     }
15112   }
15113   // pointer 事件映射 (PixiJS 7 优先使用 pointer 事件)
15114   function dispatchPointer(pointerType, rawEvent) {
15115     const touches = rawEvent.changedTouches || rawEvent.touches || [];
15116     if (!touches.length) return;
15117     const touch = touches[0];
15118     const pointerEvent = {
15119       type: pointerType,
15120       pointerId: touch.identifier || 0,
15121       pointerType: 'touch',
15122       clientX: touch.clientX,
15123       clientY: touch.clientY,
15124       pageX: touch.clientX,
15125       pageY: touch.clientY,
15126       screenX: touch.clientX,
15127       screenY: touch.clientY,
15128       x: touch.clientX,
15129       y: touch.clientY,
15130       offsetX: touch.clientX,
15131       offsetY: touch.clientY,
15132       width: 1,
15133       height: 1,
15134       pressure: pointerType === 'pointerup' ? 0 : 0.5,
15135       button: 0,
```

```

15136     buttons: pointerType === 'pointerup' ? 0 : 1,
15137     isPrimary: true,
15138     target: canvas,
15139     currentTarget: canvas,
15140     timeStamp: Date.now(),
15141     bubbles: true,
15142     cancelable: true,
15143     preventDefault() {},
15144     stopPropagation() {},
15145     stopImmediatePropagation() {},
15146   };
15147   const queue = _listeners[pointerType];
15148   if (queue) {
15149     queue.forEach(handler => {
15150       try { handler(pointerEvent); } catch (e) { console.error('[PointerEvent]', pointerType, e...
15151     });
15152   }
15153 }
15154 // 分发 window 上的全局事件 (PixiJS EventSystem 在 window 上也注册了 pointermove/pointerup)
15155 function dispatchToWindow(type, event) {
15156   if (typeof GameGlobal !== 'undefined' && GameGlobal.__windowDispatchEvent) {
15157     GameGlobal.__windowDispatchEvent(type, event);
15158   }
15159 }
15160 platform.onTouchStart((e) => {
15161   dispatch('touchstart', e);
15162   dispatchPointer('pointerdown', e);
15163   // window 上也需要收到 pointerdown
15164   var touches = e.changedTouches || e.touches || [];
15165   if (touches.length) {
15166     var t = touches[0];
15167     dispatchToWindow('pointerdown', {
15168       type: 'pointerdown', pointerId: t.identifier || 0, pointerType: 'touch',
15169       clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
15170       button: 0, buttons: 1, isPrimary: true, target: canvas,
15171       preventDefault: function() {}, stopPropagation: function() {}, stopImmediatePropagation: fu...
15172     });
15173   }
15174 });
15175 platform.onTouchMove((e) => {
15176   dispatch('touchmove', e);
15177   dispatchPointer('pointermove', e);
15178   var touches = e.changedTouches || e.touches || [];
15179   if (touches.length) {
15180     var t = touches[0];
15181     dispatchToWindow('pointermove', {
15182       type: 'pointermove', pointerId: t.identifier || 0, pointerType: 'touch',
15183       clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
15184       button: 0, buttons: 1, isPrimary: true, target: canvas,
15185       preventDefault: function() {}, stopPropagation: function() {}, stopImmediatePropagation: fu...

```

```
15186     });
15187   }
15188 });
15189 platform.onTouchEnd((e) => {
15190   dispatch('touchend', e);
15191   dispatchPointer('pointerup', e);
15192   var touches = e.changedTouches || [];
15193   if (touches.length) {
15194     var t = touches[0];
15195     dispatchToWindow('pointerup', {
15196       type: 'pointerup', pointerId: t.identifier || 0, pointerType: 'touch',
15197       clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
15198       button: 0, buttons: 0, isPrimary: true, target: canvas,
15199       preventDefault: function() {}, stopPropagation: function() {}, stopImmediatePropagation: fu...
15200     });
15201   }
15202 });
15203 platform.onTouchCancel((e) => {
15204   dispatch('touchcancel', e);
15205   dispatchPointer('pointercancel', e);
15206   var touches = e.changedTouches || [];
15207   if (touches.length) {
15208     var t = touches[0];
15209     dispatchToWindow('pointercancel', {
15210       type: 'pointercancel', pointerId: t.identifier || 0, pointerType: 'touch',
15211       clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
15212       button: 0, buttons: 0, isPrimary: true, target: canvas,
15213       preventDefault: function() {}, stopPropagation: function() {}, stopImmediatePropagation: fu...
15214     });
15215   }
15216 });
15217 // canvas.getBoundingClientRect - PixiJS 用来计算事件坐标
15218 // 必须返回逻辑像素尺寸（与触摸事件 clientX/clientY 一致），否则坐标偏移
15219 var sysInfo = platform.getSystemInfoSync();
15220 var screenW = sysInfo.screenWidth || sysInfo.windowWidth || 375;
15221 var screenH = sysInfo.screenHeight || sysInfo.windowHeight || 667;
15222 try {
15223   canvas.getBoundingClientRect = function() {
15224     return {
15225       x: 0,
15226       y: 0,
15227       top: 0,
15228       left: 0,
15229       width: screenW,
15230       height: screenH,
15231       right: screenW,
15232       bottom: screenH,
15233     };
15234   };
15235 } catch (e) {}
```

```
15236 // clientWidth/clientHeight 也需返回逻辑像素
15237 try {
15238     Object.defineProperty(canvas, 'clientWidth', { get: function() { return screenW; }, configura...
15239     Object.defineProperty(canvas, 'clientHeight', { get: function() { return screenH; }, configur...
15240 } catch (e) {}
15241 // PixiJS 会检查 canvas.style (微信 canvas 部分属性可能只读)
15242 try {
15243     if (!canvas.style) canvas.style = {};
15244     canvas.style.touchAction = "";
15245     canvas.style.msTouchAction = "";
15246     canvas.style.cursor = "";
15247     canvas.style.width = screenW + 'px';
15248     canvas.style.height = screenH + 'px';
15249 } catch (e) {}
15250 // PixiJS 检查 focus
15251 if (!canvas.focus) canvas.focus = function() {};
15252 // PixiJS EventSystem 需要 canvas.parentElement 来 addEventListener
15253 // 不能设为 null, 设为一个带事件能力的虚拟节点
15254 var _parentListeners = {};
15255 var fakeParent = {
15256     addEventListener: function(type, handler, options) {
15257         if (!_parentListeners[type]) _parentListeners[type] = [];
15258         _parentListeners[type].push(handler);
15259     },
15260     removeEventListener: function(type, handler) {
15261         if (!_parentListeners[type]) return;
15262         var idx = _parentListeners[type].indexOf(handler);
15263         if (idx !== -1) _parentListeners[type].splice(idx, 1);
15264     },
15265 };
15266 try { canvas.parentElement = fakeParent; } catch (e) {
15267     try { Object.defineProperty(canvas, 'parentElement', { value: fakeParent, configurable: true,...
15268     }
15269     try { canvas.parentNode = fakeParent; } catch (e) {
15270         try { Object.defineProperty(canvas, 'parentNode', { value: fakeParent, configurable: true, wr...
15271         }
15272     }
15273 module.exports = { TouchEvent, registerTouchEvents };
15274 /**
15275  * XMLHttpRequest 模拟
15276  * PixiJS 资源加载可能会用到
15277  */
15278 const platform = require('./platform');
15279 class XMLHttpRequest {
15280     constructor() {
15281         this.readyState = 0;
15282         this.status = 0;
15283         this.statusText = "";
15284         this.responseText = "";
15285         this.response = null;
```

```
15286   this.responseType = "";
15287   this.responseURL = "";
15288   this.withCredentials = false;
15289   this.timeout = 0;
15290   this._method = "";
15291   this._url = "";
15292   this._headers = {};
15293   this.onreadystatechange = null;
15294   this.onload = null;
15295   this.onerror = null;
15296   this.onabort = null;
15297   this.ontimeout = null;
15298   this.onprogress = null;
15299 }
15300 open(method, url) {
15301   this._method = method;
15302   this._url = url;
15303   this.readyState = 1;
15304 }
15305 setRequestHeader(key, value) {
15306   this._headers[key] = value;
15307 }
15308 getResponseHeader(key) {
15309   return this._responseHeaders ? this._responseHeaders[key.toLowerCase()] : null;
15310 }
15311 getAllResponseHeaders() {
15312   return "";
15313 }
15314 send(data) {
15315   const self = this;
15316   const responseType = this.responseType || 'text';
15317   platform.request({
15318     url: this._url,
15319     method: this._method || 'GET',
15320     header: this._headers,
15321     data: data || undefined,
15322     responseType: responseType === 'arraybuffer' ? 'arraybuffer' : 'text',
15323     dataType: responseType === 'json' ? 'json' : undefined,
15324     success(res) {
15325       self.status = res.statusCode;
15326       self.statusText = res.statusCode + "";
15327       self._responseHeaders = res.header || {};
15328       if (responseType === 'arraybuffer') {
15329         self.response = res.data;
15330       } else if (responseType === 'json') {
15331         self.response = res.data;
15332         self.responseText = JSON.stringify(res.data);
15333       } else {
15334         self.responseText = res.data;
15335         self.response = res.data;
```

```
15336     }
15337     self.readyState = 4;
15338     if (self.onreadystatechange) self.onreadystatechange();
15339     if (self.onload) self.onload();
15340 },
15341 fail(err) {
15342     self.status = 0;
15343     self.readyState = 4;
15344     if (self.onreadystatechange) self.onreadystatechange();
15345     if (self.onerror) self.onerror(err);
15346 },
15347 });
15348 }
15349 abort() {
15350     if (this.onabort) this.onabort();
15351 }
15352 addEventListener(type, handler) {
15353     this['on' + type] = handler;
15354 }
15355 removeEventListener() {}
15356 }
15357 // 静态常量
15358 XMLHttpRequest.UNSENT = 0;
15359 XMLHttpRequest.OPENED = 1;
15360 XMLHttpRequest.HEADERS_RECEIVED = 2;
15361 XMLHttpRequest.LOADING = 3;
15362 XMLHttpRequest.DONE = 4;
15363 module.exports = XMLHttpRequest;
15364 /**
15365  * Canvas 管理模块
15366  * 第一次 createCanvas() 返回主屏 canvas（小游戏环境特性，微信和抖音一致）
15367  */
15368 const platform = require('./platform');
15369 let canvas;
15370 try {
15371     canvas = platform.createCanvas();
15372 } catch (e) {
15373     console.error('[canvas] createCanvas 失败:', e);
15374     // 兜底：空对象防止后续模块加载崩溃
15375     canvas = { width: 0, height: 0, getContext: function() { return null; } };
15376 }
15377 module.exports = { canvas };
15378 /**
15379  * document 对象模拟
15380  */
15381 const platform = require('./platform');
15382 const Image = require('./Image');
15383 const { Element } = require('./element');
15384 const _eventListeners = {};
15385 const body = new Element();
```



```
15386 body.clientWidth = 0;
15387 body.clientHeight = 0;
15388 const documentElement = new Element();
15389 const document = {
15390   body,
15391   documentElement,
15392   readyState: 'complete',
15393   createElement(tag) {
15394     tag = tag.toLowerCase();
15395     switch (tag) {
15396       case 'canvas':
15397         return platform.createCanvas();
15398       case 'img':
15399       case 'image':
15400         return platform.createImage();
15401       default:
15402         return new Element();
15403     }
15404   },
15405   createElementNS(_ns, tag) {
15406     return this.createElement(tag);
15407   },
15408   createTextNode() {
15409     return new Element();
15410   },
15411   getElementById() {
15412     return null;
15413   },
15414   getElementsByTagName(tag) {
15415     if (tag === 'canvas') {
15416       const { canvas } = require('./canvas');
15417       return [canvas];
15418     }
15419     return [];
15420   },
15421   querySelector() {
15422     return null;
15423   },
15424   querySelectorAll() {
15425     return [];
15426   },
15427   // PixiJS EventSystem 使用 elementFromPoint 来确定事件目标
15428   elementFromPoint() {
15429     const { canvas } = require('./canvas');
15430     return canvas;
15431   },
15432   addEventListener(type, handler) {
15433     if (!_eventListeners[type]) _eventListeners[type] = [];
15434     _eventListeners[type].push(handler);
15435   },
```

```
15436 removeEventListener(type, handler) {
15437   if (!_eventListeners[type]) return;
15438   const idx = _eventListeners[type].indexOf(handler);
15439   if (idx !== -1) _eventListeners[type].splice(idx, 1);
15440 },
15441 dispatchEvent(ev) {
15442   const queue = _eventListeners[ev.type];
15443   if (queue) queue.forEach(handler => handler(ev));
15444 },
15445 // PixiJS 会检查这些
15446 fonts: {
15447   add() {},
15448   delete() {},
15449   has() { return false; },
15450   forEach() {},
15451 },
15452 hidden: false,
15453 visibilityState: 'visible',
15454 };
15455 module.exports = document;
15456 /**
15457  * DOM 元素模拟
15458  * 业界已知坑: HTMLCanvasElement / HTMLImageElement 必须用 constructor 直接赋值
15459  * 不能用 class extends, 否则 instanceof 校验会失败
15460  */
15461 const platform = require('./platform');
15462 class Element {
15463   constructor() {
15464     this.childNodes = [];
15465     this.style = { cursor: null };
15466     this.clientWidth = 0;
15467     this.clientHeight = 0;
15468     this._listeners = {};
15469   }
15470   appendChild(child) {
15471     this.childNodes.push(child);
15472     return child;
15473   }
15474   removeChild(child) {
15475     const idx = this.childNodes.indexOf(child);
15476     if (idx !== -1) this.childNodes.splice(idx, 1);
15477     return child;
15478   }
15479   addEventListener(type, handler) {
15480     if (!this._listeners[type]) this._listeners[type] = [];
15481     this._listeners[type].push(handler);
15482   }
15483   removeEventListener(type, handler) {
15484     if (!this._listeners[type]) return;
15485     const idx = this._listeners[type].indexOf(handler);
```

```
15486     if (idx !== -1) this._listeners[type].splice(idx, 1);
15487   }
15488   insertBefore() {}
15489   replaceChild() {}
15490   cloneNode() { return new Element(); }
15491   setAttribute() {}
15492   getAttribute() { return null; }
15493   getBoundingClientRect() {
15494     return { x: 0, y: 0, top: 0, left: 0, width: 0, height: 0, right: 0, bottom: 0 };
15495   }
15496 }
15497 // 通过 constructor 直接赋值（非 extends），确保 instanceof 正确
15498 // 鸿蒙等部分设备 createCanvas/createImage 返回对象可能无 constructor，需容错
15499 let HTMLCanvasElement, HTMLImageElement;
15500 try {
15501   const _tmpCanvas = platform.createCanvas();
15502   HTMLCanvasElement = (_tmpCanvas && _tmpCanvas.constructor) ? _tmpCanvas.constructor : Element;
15503 } catch (e) {
15504   console.warn('[element] HTMLCanvasElement 获取失败，回退到 Element:', e);
15505   HTMLCanvasElement = Element;
15506 }
15507 try {
15508   const _tmpImage = platform.createImage();
15509   HTMLImageElement = (_tmpImage && _tmpImage.constructor) ? _tmpImage.constructor : Element;
15510 } catch (e) {
15511   console.warn('[element] HTMLImageElement 获取失败，回退到 Element:', e);
15512   HTMLImageElement = Element;
15513 }
15514 class HTMLVideoElement extends Element {}
15515 module.exports = {
15516   Element,
15517   HTMLCanvasElement,
15518   HTMLImageElement,
15519   HTMLVideoElement,
15520 };
15521 /**
15522  * pixi-adapter 统一入口
15523  * 根据运行环境（真机/模拟器）将 DOM 模拟对象挂载到全局
15524  *
15525  * 关键：真机环境中 IIFE bundle 的自由变量（document、window 等）
15526  * 必须在 JS 引擎的全局作用域中可达，仅挂到 GameGlobal 不够--
15527  * GameGlobal 只是跨文件共享对象，不是全局作用域。
15528  * 因此真机需要挂到 globalThis / global 上。
15529  */
15530 var platform, noop, Image, canvas, location, document, navigator, localStorage, XMLHttpRequest, r...
15531 var Element, HTMLCanvasElement, HTMLImageElement, HTMLVideoElement;
15532 try { platform = require('./platform'); } catch(e) { console.error('[pixi-adapter] platform:', ...
15533 try { noop = require('./util').noop; } catch(e) { console.error('[pixi-adapter] util:', e); noo...
15534 try { Image = require('./Image'); } catch(e) { console.error('[pixi-adapter] Image:', e); }
15535 try { canvas = require('./canvas').canvas; } catch(e) { console.error('[pixi-adapter] canvas:',...
```

```
15536 try { location = require('./location'); } catch(e) { console.error('[pixi-adapter] location:', ...
15537 try { document = require('./document'); } catch(e) { console.error('[pixi-adapter] document:', ...
15538 try { navigator = require('./navigator'); } catch(e) { console.error('[pixi-adapter] navigator:...
15539 try { localStorage = require('./localStorage'); } catch(e) { console.error('[pixi-adapter] loca...
15540 try { XMLHttpRequest = require('./XMLHttpRequest'); } catch(e) { console.error('[pixi-adapter] ...
15541 try { registerTouchEvents = require('./TouchEvent').registerTouchEvents; } catch(e) { console.err...
15542 try {
15543   var _elem = require('./element');
15544   Element = _elem.Element;
15545   HTMLCanvasElement = _elem.HTMLCanvasElement;
15546   HTMLImageElement = _elem.HTMLImageElement;
15547   HTMLVideoElement = _elem.HTMLVideoElement;
15548 } catch(e) {
15549   console.error('[pixi-adapter] element:', e);
15550   Element = function() {};
15551   HTMLCanvasElement = Element;
15552   HTMLImageElement = Element;
15553   HTMLVideoElement = Element;
15554 }
15555 // ===== 获取真正的 JS 全局对象 =====
15556 // 优先 globalThis (ES2020+) , 其次 global (Node/V8) , 最后 GameGlobal
15557 const _realGlobal = (typeof globalThis !== 'undefined' && globalThis)
15558   || (typeof global !== 'undefined' && global)
15559   || GameGlobal;
15560 // ===== Patch Object.defineProperty =====
15561 const _origDefineProperty = Object.defineProperty;
15562 Object.defineProperty = function safeDefineProperty(obj, prop, descriptor) {
15563   try {
15564     return _origDefineProperty.call(Object, obj, prop, descriptor);
15565   } catch (e) {
15566     if (e instanceof TypeError) return obj;
15567     throw e;
15568   }
15569 };
15570 const _origDefineProperties = Object.defineProperties;
15571 Object.defineProperties = function safeDefineProperties(obj, props) {
15572   for (const key in props) {
15573     if (Object.prototype.hasOwnProperty.call(props, key)) {
15574       try {
15575         _origDefineProperty.call(Object, obj, key, props[key]);
15576       } catch (e) {
15577         if (!(e instanceof TypeError)) throw e;
15578       }
15579     }
15580   }
15581   return obj;
15582 };
15583 // ===== 获取系统信息 =====
15584 const sysInfo = platform.getSystemInfoSync();
15585 const isDevtools = sysInfo.platform === 'devtools';
```

```
15586 // ===== 定时器 & 动画帧 polyfill =====
15587 // 真机 IIFE bundle 可能无法以自由变量访问这些 API,
15588 // 在 adapter 模块作用域中它们可用, 挂到真正的全局对象。
15589 ;(function _patchTimers() {
15590   var pairs = {};
15591   if (typeof setTimeout !== 'undefined')      pairs.setTimeout = setTimeout;
15592   if (typeof clearTimeout !== 'undefined')    pairs.clearTimeout = clearTimeout;
15593   if (typeof setInterval !== 'undefined')     pairs.setInterval = setInterval;
15594   if (typeof clearInterval !== 'undefined')   pairs.clearInterval = clearInterval;
15595   if (typeof requestAnimationFrame !== 'undefined') pairs.requestAnimationFrame = requestAnima...
15596   if (typeof cancelAnimationFrame !== 'undefined') pairs.cancelAnimationFrame = cancelAnimati...
15597   for (var k in pairs) {
15598     if (typeof _realGlobal[k] === 'undefined') _realGlobal[k] = pairs[k];
15599     if (typeof GameGlobal[k] === 'undefined') GameGlobal[k] = pairs[k];
15600   }
15601 })();
15602 // ===== 禁用 OffscreenCanvas =====
15603 if (typeof GameGlobal !== 'undefined') {
15604   GameGlobal.OffscreenCanvas = undefined;
15605   _realGlobal.OffscreenCanvas = undefined;
15606 }
15607 // ===== WebGL / Canvas2D 上下文构造函数 =====
15608 // 业界已知坑: Android/鸿蒙 WebGL contextAttributes 中 stencil 返回数字 0 而不是 false,
15609 // 导致 PixiJS 判断为不支持 stencil, Filter/Mask/Graphics 全部失效
15610 let _WebGLRenderingContext = {};
15611 try {
15612   const _tmpCanvas = platform.createCanvas();
15613   if (_tmpCanvas && typeof _tmpCanvas.getContext === 'function') {
15614     // 业界经验: 优先 webgl, 不要尝试 webgl2 (鸿蒙/部分安卓返回假上下文)
15615     var _tmpGl = _tmpCanvas.getContext('webgl', {
15616       stencil: true,
15617       antialias: true,
15618       alpha: true,
15619       depth: true,
15620       preserveDrawingBuffer: true,
15621     });
15622     if (_tmpGl) {
15623       _WebGLRenderingContext = _tmpGl.constructor || {};
15624       // 业界已知坑修复: contextAttributes 中 stencil/antialias 返回 0/1 而不是 bool
15625       // PixiJS 用 === true 判断, 导致功能被误关闭
15626       try {
15627         var _origGetCtxAttr = _tmpGl.getContextAttributes;
15628         if (_origGetCtxAttr) {
15629           var _patchProto = Object.getPrototypeOf(_tmpGl);
15630           if (_patchProto) {
15631             _patchProto.getContextAttributes = function() {
15632               var attr = _origGetCtxAttr.call(this);
15633               if (attr) {
15634                 // 强制布尔化 (修复返回 0/1 的 bug)
15635                 attr.stencil = !!attr.stencil;
```

```
15636         attr.antialias = !!attr.antialias;
15637         attr.alpha = !!attr.alpha;
15638         attr.depth = !!attr.depth;
15639         attr.preserveDrawingBuffer = !!attr.preserveDrawingBuffer;
15640     }
15641     return attr;
15642 };
15643 }
15644 }
15645 } catch (e3) {
15646     console.warn('[pixi-adapter] patch getContextAttributes 失败:', e3);
15647 }
15648 // 业界已知坑: OES_vertex_array_object 在部分 Android/鸿蒙上返回假对象
15649 try {
15650     var _vaoExt = _tmpGl.getExtension('OES_vertex_array_object');
15651     if (_vaoExt && typeof _vaoExt.createVertexArrayOES !== 'function') {
15652         // 假扩展, 禁用它
15653         var _origGetExt = _tmpGl.__proto__.getExtension;
15654         _tmpGl.__proto__.getExtension = function(name) {
15655             if (name === 'OES_vertex_array_object') return null;
15656             return _origGetExt.call(this, name);
15657         };
15658         console.warn('[pixi-adapter] OES_vertex_array_object 为假扩展, 已禁用');
15659     }
15660 } catch (e4) { /* 忽略 */ }
15661 } else {
15662     console.warn('[pixi-adapter] WebGL context 获取失败 (getContext 返回 null) ');
15663 }
15664 }
15665 } catch (e) {
15666     console.warn('[pixi-adapter] WebGL 初始化异常:', e);
15667 }
15668 let _CanvasRenderingContext2D = {};
15669 try {
15670     const _tmpCanvas2 = platform.createCanvas();
15671     if (_tmpCanvas2 && typeof _tmpCanvas2.getContext === 'function') {
15672         const _tmpCtx = _tmpCanvas2.getContext('2d');
15673         if (_tmpCtx) {
15674             _CanvasRenderingContext2D = _tmpCtx.constructor || {};
15675         } else {
15676             console.warn('[pixi-adapter] Canvas2D context 获取失败');
15677         }
15678     }
15679 } catch (e) {
15680     console.warn('[pixi-adapter] Canvas2D 初始化异常:', e);
15681 }
15682 // ===== DOMParser =====
15683 class DOMParser {
15684     parseFromString() {
15685         return { documentElement: new Element() };
15686     }
15687 }
```

```
15686 }
15687 }
15688 // ===== performance =====
15689 const _performance = typeof performance !== 'undefined' ? performance : {
15690   now: Date.now.bind(Date),
15691 };
15692 // ===== window 事件系统 =====
15693 const _windowListeners = {};
15694 function _windowAddEventListener(type, handler, _options) {
15695   if (!_windowListeners[type]) _windowListeners[type] = [];
15696   _windowListeners[type].push(handler);
15697 }
15698 function _windowRemoveEventListener(type, handler) {
15699   if (!_windowListeners[type]) return;
15700   const idx = _windowListeners[type].indexOf(handler);
15701   if (idx !== -1) _windowListeners[type].splice(idx, 1);
15702 }
15703 function _windowDispatchEvent(type, event) {
15704   const queue = _windowListeners[type];
15705   if (queue) {
15706     const copy = queue.slice();
15707     copy.forEach(handler => {
15708       try { handler(event); } catch (e) { console.error('[window event]', type, e); }
15709     });
15710   }
15711 }
15712 GameGlobal.__windowDispatchEvent = _windowDispatchEvent;
15713 // ===== 事件构造函数 =====
15714 function _PointerEvent(type, opts) { this.type = type; Object.assign(this, opts || {}); }
15715 function _TouchEventCtor(type, opts) { this.type = type; Object.assign(this, opts || {}); }
15716 function _MouseEvent(type, opts) { this.type = type; Object.assign(this, opts || {}); }
15717 // ===== URL / Blob =====
15718 const _URL = {
15719   createObjectURL: function() { return ""; },
15720   revokeObjectURL: function() {},
15721 };
15722 function _Blob() {}
15723 // ===== 所有需要挂载的全局属性 =====
15724 const _allGlobals = {
15725   window: null, // 下面特殊处理
15726   document: document,
15727   navigator: navigator,
15728   location: location,
15729   Image: Image,
15730   Element: Element,
15731   HTMLCanvasElement: HTMLCanvasElement,
15732   HTMLImageElement: HTMLImageElement,
15733   HTMLVideoElement: HTMLVideoElement,
15734   WebGLRenderingContext: _WebGLRenderingContext,
15735   CanvasRenderingContext2D: _CanvasRenderingContext2D,
```

```
15736 XMLHttpRequest: XMLHttpRequest,
15737 DOMParser: DOMParser,
15738 localStorage: localStorage,
15739 performance: _performance,
15740 canvas: canvas,
15741 ontouchstart: noop,
15742 addEventListener: _windowAddEventListener,
15743 removeEventListener: _windowRemoveEventListener,
15744 self: null, // 下面特殊处理
15745 PointerEvent: _PointerEvent,
15746 TouchEvent: _TouchEventCtor,
15747 MouseEvent: _MouseEvent,
15748 URL: _URL,
15749 Blob: _Blob,
15750 };
15751 if (isDevtools) {
15752 // ===== 模拟器环境 =====
15753 // window 已存在（浏览器环境），用 defineProperty 补充/覆盖
15754 const _win = typeof window !== 'undefined' ? window : GameGlobal;
15755 for (const key in _allGlobals) {
15756 if (key === 'window' || key === 'self') continue;
15757 try {
15758 const desc = Object.getOwnPropertyDescriptor(_win, key);
15759 if (!desc || desc.configurable) {
15760 _origDefineProperty.call(Object, _win, key, { value: _allGlobals[key], configurable: true...
15761 }
15762 } catch (e) { /* 只读属性忽略 */ }
15763 }
15764 // 关键修复：包装 window.addEventListener / removeEventListener
15765 // PixiJS EventSystem 在 globalThis(window) 上注册 pointermove / pointerup,
15766 // 但 adapter 通过 _windowListeners 分发触摸事件--两个系统完全隔离。
15767 // 包装后 handler 会同时进入 _windowListeners, adapter 的 dispatchToWindow 就能触达 PixiJS。
15768 try {
15769 var _nativeWinAdd = _win.addEventListener.bind(_win);
15770 var _nativeWinRemove = _win.removeEventListener.bind(_win);
15771 var _wrappedAdd = function(type, handler, options) {
15772 _windowAddEventListener(type, handler, options);
15773 return _nativeWinAdd(type, handler, options);
15774 };
15775 var _wrappedRemove = function(type, handler, options) {
15776 _windowRemoveEventListener(type, handler);
15777 return _nativeWinRemove(type, handler, options);
15778 };
15779 // 微信新版基础库 addEventListener 可能为只读，用 defineProperty 强制覆盖
15780 _origDefineProperty.call(Object, _win, 'addEventListener', {
15781 value: _wrappedAdd, configurable: true, writable: true
15782 });
15783 _origDefineProperty.call(Object, _win, 'removeEventListener', {
15784 value: _wrappedRemove, configurable: true, writable: true
15785 });
```



```
15786 } catch (e) {
15787     console.warn('[pixi-adapter] 包装 window.addEventListener 失败:', e);
15788 }
15789 // document 属性补充
15790 try {
15791     for (const key in document) {
15792         const desc = Object.getOwnPropertyDescriptor(_win.document, key);
15793         if (!desc || desc.configurable) {
15794             _origDefineProperty.call(Object, _win.document, key, { value: document[key], configurable...
15795         }
15796     }
15797 } catch (e) { /* 忽略 */ }
15798 } else {
15799     // ===== 真机环境 =====
15800     // 关键: 必须同时挂载到 _realGlobal (JS 引擎全局对象) 和 GameGlobal (跨文件共享)
15801     // 这样 IIFE bundle 中的自由变量 document、window 等才能正确解析
15802     // window = 全局对象自身 (模拟浏览器行为)
15803     try { _realGlobal.window = _realGlobal; } catch (e) { /* 只读忽略 */ }
15804     try { GameGlobal.window = _realGlobal; } catch (e) { /* 只读忽略 */ }
15805     // self = 全局对象自身
15806     try { _realGlobal.self = _realGlobal; } catch (e) { /* 只读忽略 */ }
15807     try { GameGlobal.self = _realGlobal; } catch (e) { /* 只读忽略 */ }
15808     // addEventListener/removeEventListener 必须强制覆盖:
15809     // 微信框架可能内置了无效版本, PixiJS EventSystem 在 self 上注册
15810     // pointermove/pointerup 依赖这些函数正确工作
15811     var _forceOverwrite = ['addEventListener', 'removeEventListener'];
15812     for (const key in _allGlobals) {
15813         if (key === 'window' || key === 'self') continue;
15814         var val = _allGlobals[key];
15815         var force = _forceOverwrite.indexOf(key) !== -1;
15816         // 挂到真正的全局作用域
15817         if (force || typeof _realGlobal[key] === 'undefined') {
15818             try {
15819                 _origDefineProperty.call(Object, _realGlobal, key, { value: val, configurable: true, writ...
15820             } catch (e) {
15821                 try { _realGlobal[key] = val; } catch (e2) { /* 忽略 */ }
15822             }
15823         }
15824         // 同时挂到 GameGlobal
15825         if (force || typeof GameGlobal[key] === 'undefined') {
15826             try {
15827                 _origDefineProperty.call(Object, GameGlobal, key, { value: val, configurable: true, writ...
15828             } catch (e) {
15829                 try { GameGlobal[key] = val; } catch (e2) { /* 忽略 */ }
15830             }
15831         }
15832     }
15833 }
15834 // ===== 全局 canvas =====
15835 // 微信框架可能已将 canvas 设为只读属性, 需 try-catch 保护
```

```
15836 try { GameGlobal.canvas = canvas; } catch (e) { /* 已由框架设置 */ }
15837 try { _realGlobal.canvas = canvas; } catch (e) { /* 只读属性忽略 */ }
15838 // ===== navigator.userAgent =====
15839 try {
15840   if (_realGlobal.window && _realGlobal.window.navigator) {
15841     _realGlobal.window.navigator.userAgent = navigator.userAgent;
15842   }
15843 } catch (e) { /* 只读属性忽略 */ }
15844 // ===== 注册触摸事件 =====
15845 registerTouchEvents();
15846 /**
15847  * localStorage 模拟，基于平台 Storage API
15848  */
15849 const platform = require('./platform');
15850 const localStorage = {
15851   getItem(key) {
15852     try {
15853       return platform.getStorageSync(key) || null;
15854     } catch (e) {
15855       return null;
15856     }
15857   },
15858   setItem(key, value) {
15859     try {
15860       platform.setStorageSync(key, value);
15861     } catch (e) {
15862       console.warn('[localStorage] setItem failed:', key, e);
15863     }
15864   },
15865   removeItem(key) {
15866     try {
15867       platform.removeStorageSync(key);
15868     } catch (e) {
15869       console.warn('[localStorage] removeItem failed:', key, e);
15870     }
15871   },
15872   clear() {
15873     console.warn('[localStorage] clear() not implemented in minigame');
15874   },
15875   get length() {
15876     return 0;
15877   },
15878   key() {
15879     return null;
15880   },
15881 };
15882 module.exports = localStorage;
15883 /**
15884  * location 对象模拟
15885  */
```

```
15886 const location = {
15887   href: 'game.js',
15888   protocol: 'https:',
15889   host: 'minigame',
15890   hostname: 'minigame',
15891   port: '',
15892   pathname: '/game.js',
15893   search: '',
15894   hash: '',
15895   origin: 'https://minigame',
15896 };
15897 module.exports = location;
15898 /**
15899  * navigator 对象模拟
15900  */
15901 const platform = require('./platform');
15902 let _sysInfo;
15903 try {
15904   _sysInfo = platform.getSystemInfoSync();
15905 } catch (e) {
15906   _sysInfo = {};
15907 }
15908 // 业界核心修复：鸿蒙 platform 为 'ohos'，很多引擎只判断 android/ios
15909 // 需要在 userAgent 中统一输出 Android 标识，让 PixiJS 等框架正确识别为移动设备
15910 const _platform = _sysInfo.platform || 'unknown';
15911 const _isOHOS = _platform === 'ohos';
15912 const _system = _sysInfo.system || '';
15913 // 生成兼容 UA：鸿蒙走 Android UA（业界通用做法），确保引擎不误判为桌面
15914 let _userAgent;
15915 if (_isOHOS) {
15916   _userAgent = 'Mozilla/5.0 (Linux; Android 12; HarmonyOS; ' + (_sysInfo.model || 'HUAWEI') + ') ...
15917 } else if (_platform === 'android') {
15918   _userAgent = 'Mozilla/5.0 (Linux; Android; ' + (_sysInfo.model || '') + ') AppleWebKit/537.36 M...
15919 } else if (_platform === 'ios') {
15920   _userAgent = 'Mozilla/5.0 (iPhone; CPU iPhone OS) AppleWebKit/537.36 MiniGame PixiJS/7';
15921 } else {
15922   _userAgent = 'Mozilla/5.0 (MiniGame; ' + _platform + ') PixiJS/7';
15923 }
15924 const navigator = {
15925   platform: _isOHOS ? 'Linux armv8l' : (_sysInfo.platform || 'unknown'), // 鸿蒙伪装 Linux ARM
15926   language: _sysInfo.language || 'zh_CN',
15927   appVersion: '5.0 (MiniGame)',
15928   userAgent: _userAgent,
15929   onLine: true,
15930   maxTouchPoints: 10,
15931   vendor: '',
15932   product: '',
15933   productSub: '',
15934   hardwareConcurrency: 4,
15935   // PixiJS 可能会检查 gpu 信息
```

```
15936   gpu: ",
15937   };
15938   module.exports = navigator;
15939   /**
15940   * 平台抽象层 - 统一微信/抖音小游戏 API
15941   * 所有 adapter 模块通过此模块调用平台 API，不直接写 wx.xxx 或 tt.xxx
15942   */
15943   const _isWechat = typeof wx !== 'undefined';
15944   const _isDouyin = typeof tt !== 'undefined';
15945   const _api = _isWechat ? wx : _isDouyin ? tt : null;
15946   if (!_api) {
15947     console.error('[platform] 未检测到小游戏运行环境 (wx/tt) ');
15948   }
15949   // 安全调用包装: 防止鸿蒙等环境中 API 缺失导致崩溃
15950   function _safeCall(fn, fallback) {
15951     try {
15952       return fn();
15953     } catch (e) {
15954       console.warn('[platform] API 调用失败:', e);
15955       return fallback;
15956     }
15957   }
15958   const noop = function() {};
15959   const platform = {
15960     createCanvas: () => _api ? _api.createCanvas() : { width: 0, height: 0, getContext: function() ...
15961     createImage: () => _api ? _api.createImage() : { src: "", onload: null, onerror: null },
15962     getSystemInfoSync: () => _api ? _safeCall(() => _api.getSystemInfoSync(), { platform: 'unknown'...
15963     getStorageSync: (key) => _api ? _safeCall(() => _api.getStorageSync(key), "") : "",
15964     setStorageSync: (key, data) => _api ? _safeCall(() => _api.setStorageSync(key, data)) : undefined,
15965     removeStorageSync: (key) => _api ? _safeCall(() => _api.removeStorageSync(key)) : undefined,
15966     request: (opts) => _api ? _api.request(opts) : null,
15967     connectSocket: (opts) => _api ? _api.connectSocket(opts) : null,
15968     onTouchStart: (cb) => _api && _api.onTouchStart ? _api.onTouchStart(cb) : noop,
15969     onTouchMove: (cb) => _api && _api.onTouchMove ? _api.onTouchMove(cb) : noop,
15970     onTouchEnd: (cb) => _api && _api.onTouchEnd ? _api.onTouchEnd(cb) : noop,
15971     onTouchCancel: (cb) => _api && _api.onTouchCancel ? _api.onTouchCancel(cb) : noop,
15972     offTouchStart: (cb) => _api && _api.offTouchStart ? _api.offTouchStart(cb) : noop,
15973     offTouchMove: (cb) => _api && _api.offTouchMove ? _api.offTouchMove(cb) : noop,
15974     offTouchEnd: (cb) => _api && _api.offTouchEnd ? _api.offTouchEnd(cb) : noop,
15975     offTouchCancel: (cb) => _api && _api.offTouchCancel ? _api.offTouchCancel(cb) : noop,
15976     createInnerAudioContext: () => _api && _api.createInnerAudioContext ? _api.createInnerAudioCont...
15977     name: _isWechat ? 'wechat' : _isDouyin ? 'douyin' : 'unknown',
15978     api: _api,
15979   };
15980   module.exports = platform;
15981   /**
15982   * 工具函数
15983   */
15984   function noop() {}
15985   module.exports = { noop };
```